

Unidade IV

7 ERROS E TESTES

Em C#, falar de erros significa lidar com diferentes camadas: aquilo que o compilador consegue detectar, aquilo que só aparece em tempo de execução e aquilo que é simplesmente um raciocínio errado do desenvolvedor.

Os chamados **erros de compilação** surgem quando o código viola regras da linguagem ou do sistema de tipos: um identificador inexistente, uma conversão impossível, uma instrução malformada etc. O compilador do C# analisa o código-fonte, verifica a conformidade com a especificação da linguagem e, se encontrar um problema, não gera o executável. Esses diagnósticos funcionam como uma barreira inicial: impedem que o programa chegue a executar em situações em que a linguagem já consegue enxergar a inconsistência de forma estática.

Quando o código compila, ainda resta uma ampla categoria de problemas que só aparece em tempo de execução. É aqui que entram as **exceções**, objetos derivados de `System.Exception` que representam falhas detectadas durante a execução do programa. Ao acessar um elemento fora dos limites de um array, o runtime lança `IndexOutOfRangeException`; ao desreferenciar uma variável nula, surge `NullReferenceException`; ao tentar dividir por zero, aparece `DivideByZeroException`; problemas de entrada/saída (I/O, do inglês input/output) típicos resultam em subclasses de `IOException`.

Cada uma dessas exceções carrega propriedades que ajudam na análise: `Message` com a descrição do erro, `StackTrace` com a pilha de chamadas no momento da falha e, em muitos casos, `InnerException` para preservar a causa original quando uma exceção é encapsulada em outra de nível mais alto.

Como veremos em detalhes, o mecanismo de tratamento em C# é baseado em blocos `try`, `catch` e `finally`. Um trecho de código que pode falhar é envolvido em `try`; se alguma instrução ali lançar uma exceção, o runtime interrompe o fluxo normal, percorre a pilha até encontrar um `catch` compatível com o tipo da exceção e transfere o controle para esse ponto.

Dentro do `catch`, o desenvolvedor pode registrar logs, notificar camadas superiores, traduzir a exceção em um tipo mais aderente ao domínio da aplicação ou simplesmente repropagar o problema com `throw`.

O bloco `finally`, quando presente, é executado independentemente de ter ocorrido ou não uma exceção e costuma ser usado para liberar recursos. Versões modernas da linguagem acrescentam filtros de exceção com a palavra-chave `when`, permitindo que um `catch` só seja considerado se uma

condição booleana adicional for verdadeira, o que ajuda a separar cenários aparentados sem fragmentar o código em múltiplos blocos quase idênticos.

Há uma distinção conceitual importante entre erros que representam **violações de contrato** e situações que **pertencem ao fluxo normal** do domínio. Entrada de usuário inválida, ausência de um registro em banco de dados e cancelamento explícito de uma operação são casos previsíveis: o sistema sabe, de antemão, que podem ocorrer.

Nesses cenários, costuma ser mais claro modelar o problema com lógica de controle tradicional (`if`, `switch`) e tipos de retorno adequados, como o padrão `TryParse` na BCL, em que métodos retornam um valor booleano indicando sucesso e evitam lançar exceções em condições comuns.

Já exceções em sentido forte deveriam sinalizar estados em que alguma suposição interna foi quebrada ou em que o ambiente impôs uma condição na qual não faz sentido prosseguir como se nada tivesse acontecido, como corrupção de dados em disco, perda súbita de conectividade em um ponto crítico de uma transação ou acesso a estruturas internas em um estado que a própria implementação considerava impossível.

A divisão por zero é um exemplo didático dessa diferença de perspectivas. No momento em que o programa recebe um denominador vindo de entrada externa, o valor zero é um cenário previsível; o código pode testar o denominador e tratar o caso de forma controlada, devolvendo uma mensagem adequada ou pedindo nova entrada.

A exceção `DivideByZeroException` entra em cena quando esse cuidado não foi tomado em algum ponto da cadeia e a operação aritmética acaba sendo invocada com um denominador inválido, violando o contrato implícito de que esse valor deveria ter sido filtrado antes.

Do ponto de vista da linguagem e do runtime, faz sentido que essa condição seja descrita como um erro em tempo de execução; do ponto de vista do design da aplicação, é desejável que o uso de exceções não substitua a validação explícita de situações esperadas. Além dos erros de compilação e das exceções, existe uma classe de falhas que não dispara mensagens do compilador nem gera `Exception`, mas prejudica diretamente o resultado: **os erros de lógica**.

Um cálculo de juros implementado com fórmula incorreta, uma condição de fronteira ignorada em uma regra tributária, um laço que processa um item a menos ou a mais, tudo isso pode produzir saídas enganadoramente normais, sem qualquer sinal de falha técnica.

Nesse tipo de problema, o código passa por todas as verificações da linguagem e do runtime, porém está em desacordo com os requisitos do domínio. Em C#, assim como em outras linguagens, a principal defesa contra esse tipo de erro está em **testes bem projetados**, revisão de código e especificações claras, mais do que em mecanismos automáticos da linguagem.



Observação

A construção de aplicações C# robustas em relação a erros costuma combinar tais peças: o compilador filtrando erros estruturais, o sistema de exceções registrando falhas em tempo de execução, a distinção cuidadosa entre situações tratáveis como parte do fluxo normal e estados verdadeiramente excepcionais e, por cima disso, um conjunto de testes e ferramentas de observabilidade que ajuda a detectar tanto as violações técnicas quanto os desvios de lógica de negócio.

O modo como o desenvolvedor escolhe declarar pré-condições, validar entradas e lançar e capturar exceções influencia diretamente a legibilidade do código e a facilidade de diagnosticar problemas durante a manutenção.

Uma abordagem estruturada à qualidade de software começa com testes automatizados robustos. Assim, é importante destacar que a testagem automatizada é um elemento fundamental do desenvolvimento ágil. Logo, **escrever testes unitários** é uma prática essencial para manter a confiabilidade do código.

Quando o desenvolvimento se apoia fortemente em integração contínua e práticas DevOps modernas, **integrar testes já nas fases iniciais do ciclo de vida** do software torna-se essencial para a entrega contínua de software confiável.

DevOps é um conjunto de práticas e princípios que visam melhorar a colaboração entre as equipes de desenvolvimento e de operações de TI. Ele se tangibiliza em um esforço colaborativo e multidisciplinar dentro de uma organização para automatizar a entrega contínua de novas versões de software, enquanto é possível garantir correção e confiabilidade.



Saiba mais

Para se aprofundar no assunto, leia os livros a seguir.

KIM, G. *et al.* *The DevOps handbook: how to create world-class agility, reliability, & security in technology organizations*. 2. ed. Portland: IT Revolution, 2021.

FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps – building and scaling high performing technology organizations*. Portland: IT Revolution, 2018.

Testes automatizados ajudam a capturar os casos de fronteira que poderiam passar despercebidos em testes manuais ou no próprio uso da aplicação. O problema de montar toda a aplicação e testá-la como um todo é que é muito fácil perder casos de fronteira ou cenários não antecipados.

Os testes unitários forçam o desenvolvedor a pensar em cada unidade isoladamente e verificar essas unidades de forma independente. Em outras palavras, sem a disciplina de testes de unidade, aumentam-se os riscos de regressões e bugs ocultos.



Observação

Quando o código é alterado, deseja-se verificar se funcionalidades que já funcionavam continuam corretas, isto é, se não houve regressão. Em vez de um tipo separado, costuma-se chamar de regressão o conjunto de testes que são reexecutados depois de uma modificação. A própria suíte de testes unitários, de integração, de sistema e de aceitação, executada continuamente em pipelines de integração contínua (`dotnet test` em GitHub Actions, Azure DevOps e afins), atua como suíte de regressão.

Um caso comum é o de um bug específico: primeiro se reproduz o erro em um teste, depois se corrige o código para que o teste passe. Esse teste passa a fazer parte do conjunto de regressão e ajuda a impedir que o mesmo problema volte a aparecer.

Entre as ferramentas de testes para C# e .NET, o framework xUnit ocupa lugar de destaque, sendo o mais popular no ecossistema moderno. Diferentemente de abordagens mais antigas, como MSTest, o xUnit foi desenvolvido especificamente para .NET Core e .NET 5/6/7/8, favorecendo simplicidade e extensibilidade. Em projetos .NET, a utilização do xUnit é praticamente padrão para testes de integração e unidade.

A filosofia do xUnit se alinha a práticas ágeis, de modo que os testes são tratados como código de primeira classe, escritos da mesma forma que o código de produção. Em termos conceituais, um teste no xUnit segue quatro fases básicas: configuração de fixture, execução do código em teste, verificação do resultado e desmontagem dos recursos (teardown).

Esse ciclo reflete uma boa estratégia de testes: prepara-se o ambiente (constrói-se o objeto, inicializa se banco em memória etc.), chama-se o código que se quer verificar, compara-se o resultado esperado com o real e, por fim, limpa-se tudo que foi utilizado no teste. Neste tópico, veremos o uso do xUnit em detalhes.

7.1 Exceções, hierarquia, tratamento e criação de exceções

O tratamento de erros e condições excepcionais é realizado por meio de **exceções**, que são objetos especiais lançados quando o programa encontra problemas em tempo de execução.



Lembrete

Em oposição às exceções, vale lembrar que os erros de compilação decorrem da violação de regras da linguagem ou do sistema de tipos, como um identificador inexistente ou uma instrução malformada.

Já os erros de lógica produzem saídas sem qualquer sinal de falha técnica, mas que estão em desacordo com os requisitos do domínio, em virtude, por exemplo, da implementação de uma fórmula incorreta para o cálculo de juros.

Diferentemente de códigos que usam retornos de erro para controle de fluxo, o mecanismo de exceções separa claramente o código de identificação de erros do código de tratamento de erros. Isso significa que um bloco de código pode simplesmente lançar (`throw`) uma exceção quando ocorrer uma situação inesperada, e outro trecho de código (geralmente em um `try/catch`) se encarrega de capturá-la e tratá-la, mantendo o código principal limpo e focado em seu propósito principal.

Em C#, as exceções são propagadas automaticamente pela pilha de chamadas até que um bloco `catch` correspondente seja encontrado; caso contrário, elas terminam a aplicação com uma mensagem de erro.

No .NET, as exceções possuem uma **hierarquia de classes** que permite distinguir diferentes tipos de erros. A classe raiz de toda exceção em .NET é `System.Exception`, que representa genericamente qualquer erro de execução. Além dela, existem as classes `System.SystemException` (base para exceções do sistema, como `NullReferenceException`, `IndexOutOfRangeException`) e `System.ApplicationException`, definida para exceções criadas pelo programador.

Na prática, toda exceção específica – seja do sistema ou do aplicativo – deriva de `System.Exception`, que é a classe base de todas as exceções no .NET Framework, e `System.ApplicationException` é a classe base para todas as exceções específicas de aplicação. Isso significa que, ao lançar uma exceção (por exemplo, `throw new Exception("mensagem")`), o tipo dessa exceção pode ser capturado por blocos `catch` específicos ou genéricos, conforme a hierarquia permitir.

De fato, o bloco `catch` sem especificação de tipo captura `System.Exception` por padrão, mas podemos criar vários blocos `catch` em sequência para tratar cada exceção de forma diferente, seguindo a hierarquia de tipos.

Para capturar exceções em C#, utilizam-se as estruturas `try`, `catch` e `finally`. O **código propenso a erros** é colocado no bloco `try`, enquanto um ou mais blocos `catch` seguintes tratam exceções lançadas. Opcionalmente, um bloco `finally` executa sempre, ocorrendo exceção ou não, sendo ideal para liberar recursos (arquivos, conexões etc.).

Em outras palavras, um bloco `try-catch` é usado para capturar e tratar exceções em C#. O bloco `try` contém o código que pode lançar uma exceção, e o bloco `catch` contém o código que a trata. Por exemplo:

```
try
{
    // Código que pode lançar exceções
    int resultado = Numerador / Denominador;
    Arquivo.Write("Salvando dados...");
}
catch (DivideByZeroException ex)
{
    // Tratamento específico de divisão por zero
    Console.WriteLine($"Erro de divisão: {ex.Message}");
}
catch (IOException ex)
{
    // Tratamento de erros de I/O
    Console.WriteLine($"Erro de I/O: {ex.Message}");
}
catch (Exception ex)
{
    // Tratamento genérico de outras exceções
    Console.WriteLine($"Erro inesperado: {ex.Message}");
}
finally
{
    // Código sempre executado (por exemplo, liberar recursos)
    Console.WriteLine("Operação finalizada.");
}
```

Nesse exemplo, cada tipo de exceção (`DivideByZeroException`, `IOException` etc.) é capturado por seu bloco `catch` específico, evitando-se assim o uso de um único `catch` (`Exception`) genérico de modo desnecessário. O bloco `finally` mostra um uso típico: independentemente de erro ou não, a mensagem de finalização é exibida, representando operações como fechar arquivos ou liberar memória. Essa separação permite tratar cada situação de erro adequadamente.

O tratamento de exceções inclui envolver o código propenso a erro em um bloco `try`, capturar todas as exceções, capturar exceções específicas e usar filtros. No C#, além do `try/catch/finally`, temos filtros de exceção com a cláusula `when`, que permite testar condições adicionais no momento da captura. Veja a seguir:

```
try
{
    ProcessarArquivo(caminhoArquivo);
}
catch (FileNotFoundException ex) when (caminhoArquivo.EndsWith(".txt"))
{
    Console.WriteLine("Arquivo de texto não encontrado. Crie o arquivo antes de executar.");
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"Arquivo não encontrado: {ex.FileName}");
}
```

Nesse trecho, o primeiro **catch** utiliza um filter (**when**) para tratar especificamente a ausência de arquivos de texto (.txt) de forma personalizada; o segundo bloco captura outras falhas de arquivo padrão. Isso evita duplicação de código e mantém o tratamento claro.

Além das exceções predefinidas pelo .NET, os desenvolvedores podem criar exceções personalizadas derivadas de `Exception` (ou `ApplicationException`). Essas classes customizadas permitem sinalizar erros específicos do domínio da aplicação. Por exemplo:

```
// Definição de exceção personalizada
public class UsuarioInvalidoException : Exception
{
    public UsuarioInvalidoException(string mensagem)
        : base(mensagem) { }

    public UsuarioInvalidoException(string mensagem, Exception innerException)
        : base(mensagem, innerException) { }
}

// Lançando a exceção personalizada
if (usuario == null)
    throw new UsuarioInvalidoException("O usuário não pode ser nulo.");
```

Nesse código, a classe `UsuarioInvalidoException` herda de `Exception` e repassa a mensagem de erro à classe base. É comum também implementar construtores que aceitem uma `InnerException`, para encadear erros internos.

Embora exista a classe `ApplicationException` como raiz para exceções de negócio, a prática atual recomenda herdar diretamente de `Exception`, conforme a hierarquia do .NET. Note-se que a exceção personalizada permite lançar e capturar um erro de forma mais semântica, distinguindo, por exemplo, argumentos inválidos, estado impróprio do objeto etc.

Toda instância de `System.Exception` fornece propriedades que ajudam no diagnóstico. A propriedade `Message` contém a descrição do erro fornecida ao lançar a exceção; `StackTrace` informa a pilha de chamadas no momento em que a exceção ocorreu; e `InnerException` referencia outra exceção interna, caso a atual tenha sido gerada em razão de outra. Vamos fazer um exemplo de uso:

```
try
{
    ConectarBanco();
}
catch (Exception ex)
{
    Console.WriteLine($"Erro: {ex.Message}");
    Console.WriteLine($"Origem: {ex.StackTrace}");
    if (ex.InnerException != null)
    {
        Console.WriteLine($"Erro interno: {ex.InnerException.Message}");
    }
}
```

Aqui, `ex.Message` pode conter, por exemplo, Falha ao conectar ao banco de dados e `ex.StackTrace` poderia ter:

```
em MeuProjeto.Banco.ConectarBanco()
em MeuProjeto.Program.Main(String[] args)
```

Nesse trecho, extraímos a mensagem e a origem do erro para log ou exibição, e verificamos `InnerException` para detalhes adicionais. Isso é útil, por exemplo, quando um método captura uma exceção e lança outra, encadeando as informações. Em cenários reais, essas propriedades orientam o desenvolvedor ou a equipe de suporte a entenderem em qual local do código e por que o problema ocorreu. A seguir, temos um exemplo ilustrativo do conteúdo de `ex.InnerException`:

```
Erro interno: Timeout expirado. O tempo de espera para a operação foi excedido.
```

Vimos que a cláusula **when** introduz filtros de exceção no C# e permite avaliar uma condição extra no **catch**, sem precisar duplicar o bloco. Em essência, o filtro garante que um **catch** só seja executado se a expressão após **when** for verdadeira.

No exemplo anterior, usamos **when** (`ex.Message.Contains(...)`) ou testamos atributos do erro, refinando o tratamento. Essa abordagem mantém o código de **catch** enxuto e evita código duplicado, especialmente quando queremos diferenciar situações similares. Os filtros são avaliados no momento do lançamento da exceção, podendo até mesmo não capturar a exceção naquele **catch** específico e deixá-la propagar para outro bloco ou para o runtime, caso a condição falhe.

Ao usar exceções, há práticas recomendadas que garantem clareza e desempenho. Em primeiro lugar, **não se deve usar exceções como alternativa ao fluxo normal de controle**; exceções têm um **custo de desempenho** e devem representar **erros reais**, não eventos esperados (por exemplo, é melhor usar laços ou verificações de condição do que lançar e capturar uma exceção em cada interação).



Observação

Erro real não significa algo que ninguém jamais poderia prever, e sim a violação de um contrato, de um pressuposto ou de um invariante que o código considerava garantido. Às vezes, isso é consequência de um descuido ao não tratar um evento esperado, como não validar a entrada do usuário; em outros casos vem de condições externas e pouco controláveis, como disco cheio, falha de rede, corrupção de arquivo ou um bug lógico interno.

Deve-se capturar somente exceções específicas em vez de `catch(Exception)` genérico, para não ocultar erros inesperados; se usar um `catch` genérico, o ideal é relançar a exceção (com `throw`), após registrar ou processar informações relevantes, para não perder o rastreo original. Por exemplo:

```
try
{
    ProcessarPagamento();
}
catch (PagamentoRecusadoException ex)
{
    LogErro(ex);
    // Decide não relançar, pois foi tratada localmente
}
catch (Exception)
{
    // Relança para tratamento global
    throw;
}
```

Percebe-se que `PagamentoRecusadoException` foi tratada especificamente, enquanto outros erros foram relançados para camadas superiores. Além disso, é importante usar o `finally` ou a instrução `using` para liberar recursos (conexões, arquivos etc.) sempre, mesmo em caso de erro. O uso de `using(var recurso = ...) { ... }` garante que `Dispose` seja chamado de maneira automática, o que evita vazamentos de memória e inconsistências. Também não é recomendado sufocar exceções silenciosamente (por exemplo, capturar e não fazer nada no `catch`), pois isso dificulta a depuração.

Em síntese, deve-se usar o mecanismo de exceções com responsabilidade: lança-las apenas em situações realmente excepcionais, tratá-las de forma granular, limpar/liberar os recursos adequadamente e relançar as que não puder tratar por completo. Essas práticas tornam o código mais robusto, legível e alinhado às diretrizes de C# e .NET para tratamento de erros.

7.2 Testes com xUnit e uso do Gerenciador de Testes

O framework xUnit.net é uma das ferramentas de testes de unidade mais utilizadas em projetos C#. Trata-se de um framework de testes criado pelos autores originais do NUnit, mas que adota uma abordagem mais simples e minimalista em comparação com outros frameworks. O xUnit concentra-se em ter menos características, e não mais do que outras ferramentas, adotando convenções próprias (por exemplo, construtores de classe de teste em vez dos métodos tradicionais de setup/teardown).

Cada método de teste é marcado com atributos especiais: **[Fact]** para testes sem parâmetros fixos (assertivas invariantes) e **[Theory]** para testes parametrizados com dados de entrada diversos. O uso desses atributos distingue o xUnit de frameworks legados, posicionando cada teste como um fato verdadeiro ou uma teoria que só vale sob certas condições. No desenvolvimento orientado a testes (TDD, do inglês test-driven development), por exemplo, começa-se escrevendo um teste que falha e depois se implementa o código de produção para fazê-lo passar.

A integração do xUnit com o Visual Studio facilita esse fluxo: ao criar um projeto de teste, geralmente se usa o comando `dotnet new xunit` ou o assistente da IDE, que já adiciona automaticamente referências essenciais, como `xunit`, `xunit.runner.visualstudio` e `Microsoft.NET.Test.Sdk`. Esses pacotes permitem que o Gerenciador de Testes do Visual Studio descubra e execute os testes. Em um arquivo de projeto típico (por exemplo, o `.csproj` de um projeto de testes xUnit), veremos trechos como:

```
<ItemGroup>
  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="..." />
  <PackageReference Include="xunit" Version="..." />
  <PackageReference Include="xunit.runner.visualstudio" Version="..." />
</ItemGroup>
```

Essas referências garantem que o **runner** de testes do Visual Studio seja configurado para o xUnit. Sem elas, o Gerenciador de Testes não encontraria os métodos de teste definidos nas classes.



Observação

No contexto do xUnit, o runner é o componente (ferramenta) responsável por descobrir, executar e registrar o resultado dos testes. Ele examina o assembly de testes, procura métodos marcados com atributos como **[Fact]** e **[Theory]**, instancia as classes de teste quando necessário, chama os métodos de teste, captura exceções, contabiliza quantos testes passaram, falharam ou foram ignorados e, por fim, mostra esse resumo no console ou no Gerenciador de Testes.

Uma vez configurado o projeto de teste, escrevem-se as classes de teste e os métodos de teste. Em xUnit, cada classe de teste pode agrupar testes relacionados, geralmente seguindo o padrão de uma classe de teste para cada classe de código de produção.

Cada método de teste recebe o atributo `[Fact]` quando representa um cenário isolado, ou `[Theory]` acompanhado de atributos `[InlineData]` para testar múltiplos casos. Por exemplo, suponha que temos uma classe `Calculadora` com o método `Somar(int a, int b)`. Escreveríamos testes assim:

```
public class CalculadoraTests
{
    [Fact]
    public void Somar_2e2_Retorna4()
    {
        int resultado = Calculadora.Somar(2, 2);
        Assert.Equal(4, resultado);
    }
}
```

O atributo `[Fact]` indica ao runner que o método é um teste que deve passar invariavelmente. Para validar comportamentos com vários valores, usa-se `[Theory]` e múltiplos `[InlineData]`. Ao testar números ímpares, teríamos o que segue:

```
public class NumeroTests
{
    [Theory]
    [InlineData(3)]
    [InlineData(4)]
    [InlineData(5)]
    public void EhImpar_DeveRetornarTrueApenasParaImpares(int valor)
    {
        bool ehImpar = (valor % 2) == 1;
        Assert.True(ehImpar == (valor % 2 == 1));
    }
}
```

Nesse caso, o xUnit executa o método de teste uma vez para cada conjunto de dados fornecido, reportando qual entrada falhou ou passou no teste. Quando executamos `dotnet test` ou clicamos em Executar Todos os Testes no Gerenciador de Testes, o console (ou o próprio Gerenciador de Testes) mostrará resultados como `Discovered: UnitTests... Starting:... Finished...`, incluindo contagem de testes total e falhas. Vale notar que, mesmo com três chamadas de `InlineData`, o runner trata cada combinação como um teste distinto, relatando falhas detalhadamente para cada parâmetro.

No xUnit, não há métodos estáticos `[SetUp]` ou `[TearDown]`, em vez disso, a própria classe de teste funciona como o contexto. O framework cria uma nova instância da classe de teste antes de cada método marcado com `[Fact]` ou `[Theory]`. Portanto, qualquer código colocado no construtor da classe de teste é executado antes de cada teste, servindo como setup automático. Do mesmo modo, se a classe implementar `IDisposable`, o método `Dispose()` será chamado após cada teste, permitindo cleanup. Assim, um padrão comum é:

```
public class RecursoTests : IDisposable
{
    private Recurso _recurso;
```

```
public RecursoTests()
{
    _recurso = new Recurso();
    // Código de inicialização, se preciso
}

[Fact]
public void TestarAlgoComRecurso()
{
    // Usa _recurso em algum teste
}

public void Dispose()
{
    _recurso.Dispose();
}
}
```

Isso equivale à funcionalidade tradicional de setup/teardown, sem usar atributos estáticos.

Outra característica do xUnit é a variedade de assertions disponíveis na classe `Assert`, que são métodos estáticos para verificar condições – por exemplo, `Assert.Equal(expected, actual)`, `Assert.True(condição)`, `Assert.Throws<T>(...)` para exceções esperadas etc. Esses métodos geram uma falha de teste se a condição não for atendida. Os textos de erro e stack trace são automaticamente exibidos no Gerenciador de Testes em caso de falha, facilitando o diagnóstico.

Vamos fazer um exemplo mais completo: considere um aplicativo de relacionamentos. Criaremos uma classe **Perfil** para representar cada usuário, com atributos como nome, idade, lista de interesses e localização.

```
public class Perfil {
    public string Nome { get; set; }
    public int Idade { get; set; }
    public List<string> Interesses { get; set; }
    public string Localizacao { get; set; }

    private List<Perfil> perfisCurtidos = new List<Perfil>();
    private List<Perfil> matches = new List<Perfil>();

    public Perfil(string nome, int idade, List<string> interesses, string
localizacao) {
        Nome = nome;
        Idade = idade;
        Interesses = interesses;
        Localizacao = localizacao;
    }
    // ...
}
```

Esse código cria um perfil com propriedades automáticas para os atributos básicos. Usamos `List<string>` para os interesses, pois `List<T>` é uma coleção genérica dinâmica que permite adicionar elementos em tempo de execução. Já no construtor, inicializamos esses campos.

Para implementar a função de curtir (like) e detectar match, definimos um método como **Curtir(Perfil outro)**, que adiciona o outro perfil à lista de curtidas e verifica se ele já havia curtido reciprocamente (condição de match). Em caso de match, adicionamos cada perfil à lista de matches do outro. Veja a seguir:

```
public bool Curtir(Perfil outro) {
    if (outro == null) return false;
    if (!perfisCurtidos.Contains(outro)) {
        perfisCurtidos.Add(outro); // Adiciona o perfil curtido
    }
    // Verifica se houve curtida mútua: se outro já curtiu este
    if (outro.perfisCurtidos.Contains(this)) {
        if (!matches.Contains(outro)) {
            matches.Add(outro);
            outro.matches.Add(this);
        }
        return true; // Match detectado
    }
    return false; // Ainda não houve match
}

public bool TemMatchCom(Perfil outro) {
    return matches.Contains(outro);
}
```

Aqui, o método **Curtir** retorna true se ocorrer um match mútuo. A detecção de interesses comuns pode ser feita com LINQ (Intersect), que produz a interseção de duas sequências. Adicionamos ainda uma função de compatibilidade (EhCompatívelCom) baseada em interesses e localização. Por exemplo, podemos exigir pelo menos um interesse em comum e mesma cidade:

```
public bool EhCompatívelCom(Perfil outro) {
    if (outro == null) return false;
    bool interessesComuns = this.Interesses.Intersect(outro.Interesses).Any();
    bool mesmaLocal = string.Equals(this.Localizacao, outro.Localizacao,
StringComparison.OrdinalIgnoreCase);
    return interessesComuns && mesmaLocal;
}
```

Nesse trecho, **Intersect** retorna os interesses em comum. Assim determinamos se dois perfis compartilham tópicos de interesse e localização igual. Usaremos o xUnit para testar a lógica de match. Primeiramente, vamos criar a solução e o projeto principal (biblioteca de classes). O passo a passo é:

- Abra o Visual Studio 2026.
- Menu Arquivo > Novo > Projeto.
- Filtro de linguagem: C# (figura 16).
- Escolha o template Biblioteca de Classes (figura 16).
- Nome do projeto: TinderApp (figura 17).

- Escolha um Framework .NET (figura 18).
- Confirme a criação do projeto.

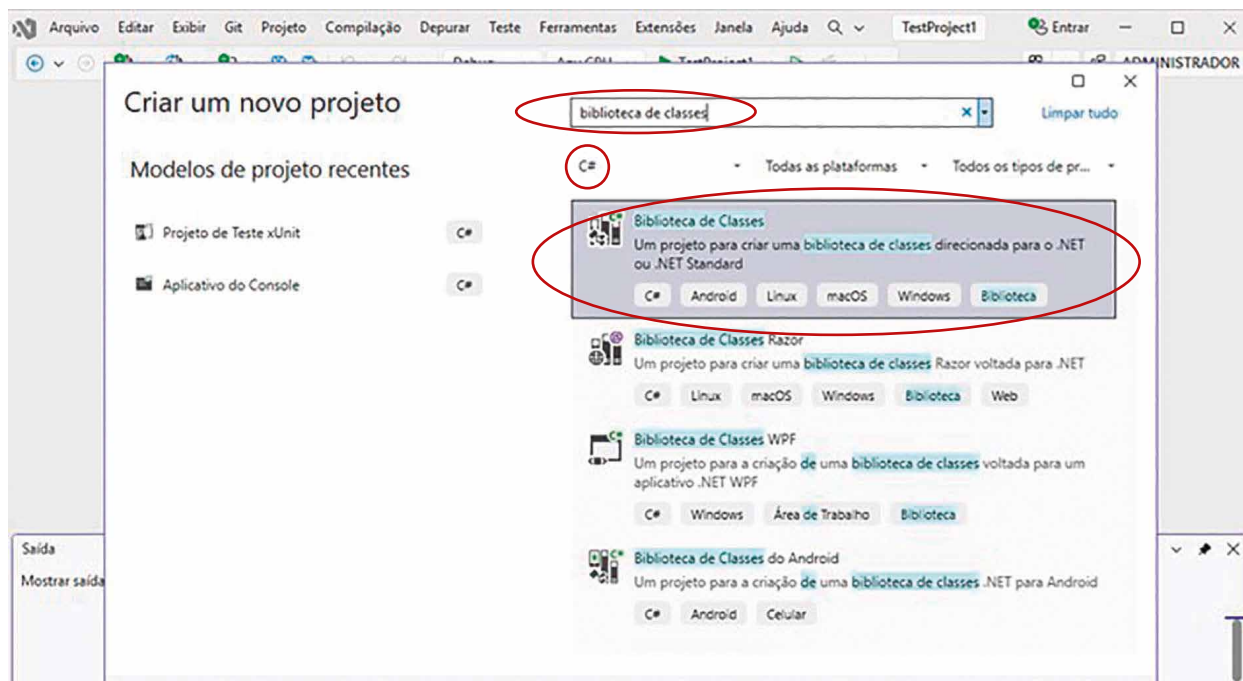


Figura 16 – Criando um novo projeto do tipo Biblioteca de Classes

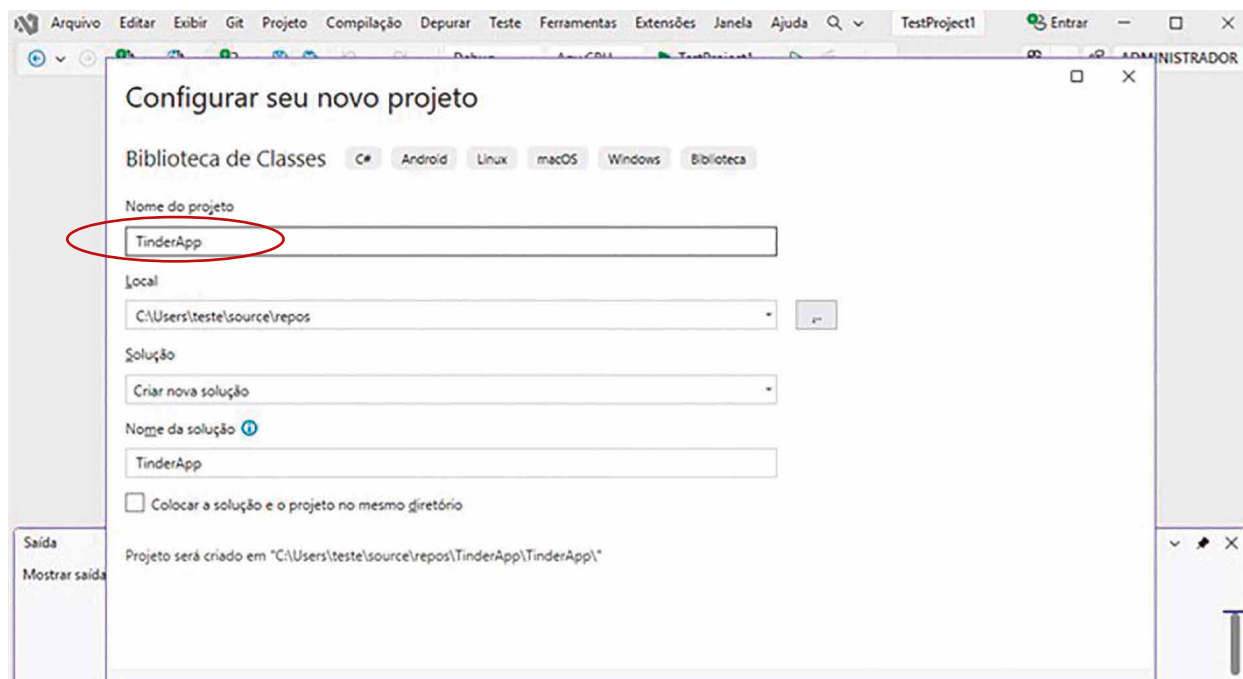


Figura 17 – Nomeando o projeto como TinderApp

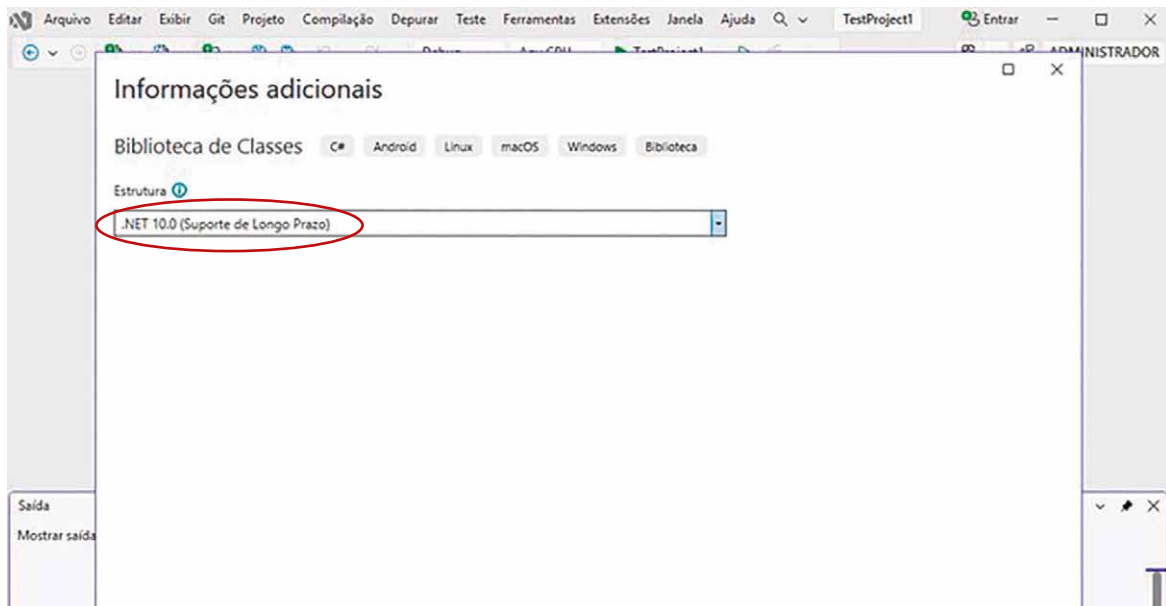


Figura 18 – Escolhendo o Framework

Agora, vamos implementar a classe Perfil no projeto TinderApp. Veja os passos a seguir:

- No Gerenciador de Soluções, clique com o botão direito no projeto TinderApp (figura 19). Caso o Gerenciador de Soluções não esteja visível, clique no menu Exibir > Gerenciador de Soluções.
- Selecione Adicionar > Classe... (figura 19).
- Nomeie a classe como Perfil.cs (figura 20).

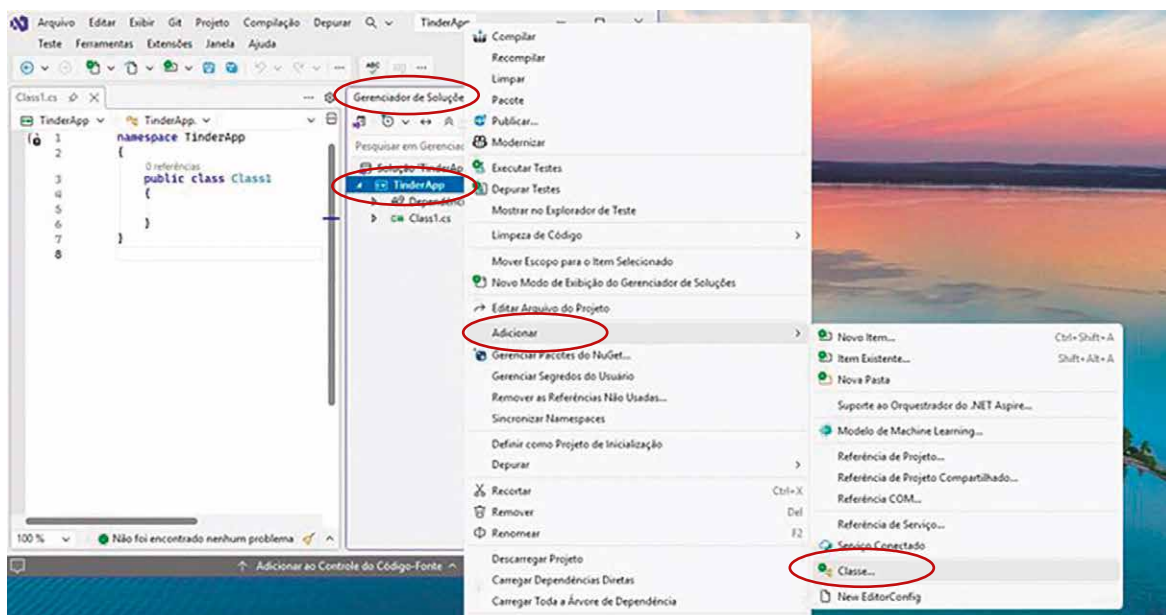


Figura 19 – Inserindo a classe Perfil.cs

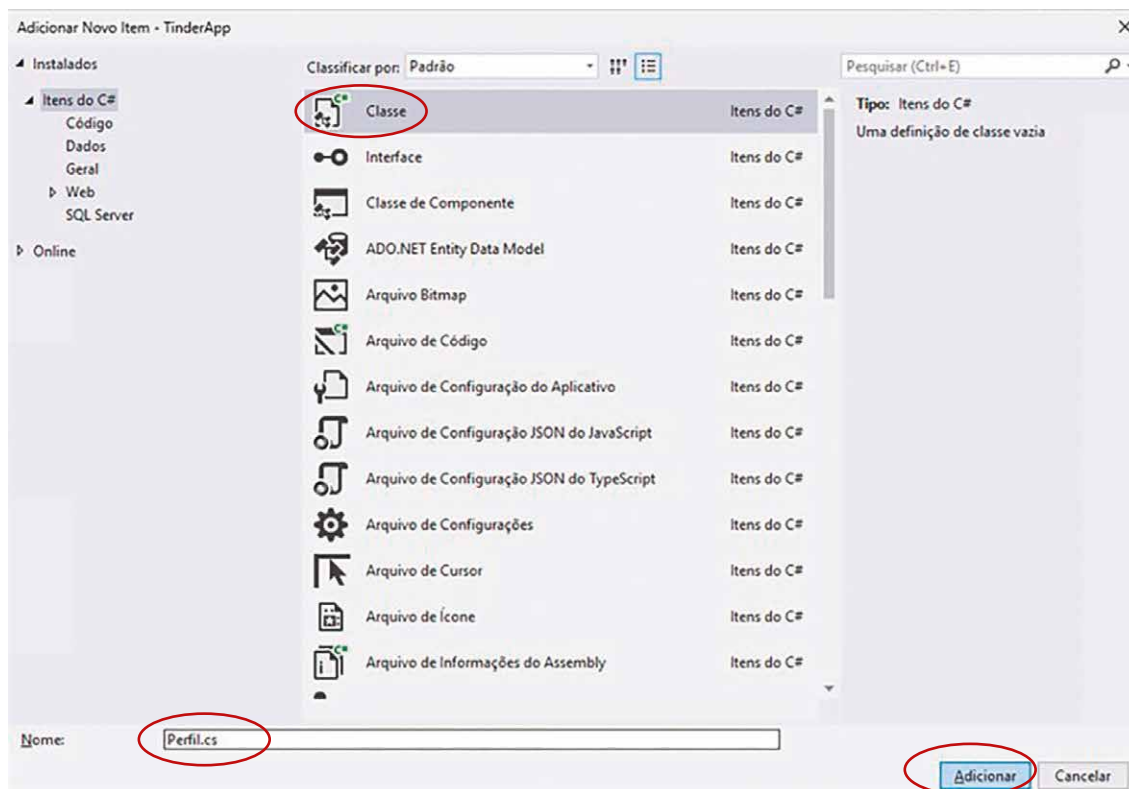


Figura 20 – Nomeando a classe Perfil.cs

Substitua o conteúdo gerado automaticamente no editor, em Perfil.cs, pelo código completo a seguir:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace TinderApp
{
    public class Perfil
    {
        public string Nome { get; set; }
        public int Idade { get; set; }
        public List<string> Interesses { get; set; }
        public string Localizacao { get; set; }

        private readonly List<Perfil> perfisCurtidos = new List<Perfil>();
        private readonly List<Perfil> matches = new List<Perfil>();

        public Perfil(string nome, int idade, List<string> interesses, string localizacao)
        {
            Nome = nome;
            Idade = idade;
            Interesses = interesses ?? new List<string>();
            Localizacao = localizacao;
        }
    }
}
```



```
/// <summary>
/// Registra uma curtida no perfil "outro" e retorna true
/// se houver curtida mútua (match).
/// </summary>
public bool Curtir(Perfil outro)
{
    if (outro == null)
        return false;

    // Adiciona o outro perfil à lista de curtidos, se ainda não estiver

    if (!perfisCurtidos.Contains(outro))
    {
        perfisCurtidos.Add(outro);
    }

    // Verifica se o outro perfil já havia curtido este
    if (outro.perfisCurtidos.Contains(this))
    {
        // Garante que o match seja registrado nos dois perfis
        if (!matches.Contains(outro))
        {
            matches.Add(outro);
        }

        if (!outro.matches.Contains(this))
        {
            outro.matches.Add(this);
        }

        return true; // houve match
    }

    return false; // ainda não houve match mútuo
}

/// <summary>
/// Indica se este perfil tem match com o perfil "outro".
/// </summary>
public bool TemMatchCom(Perfil outro)
{
    if (outro == null)
        return false;

    return matches.Contains(outro);
}

/// <summary>
/// Critério simples de compatibilidade: pelo menos um interesse
/// em comum e mesma localização (comparação sem diferenciar maiúsculas/
minúsculas).
/// </summary>
public bool EhCompatívelCom(Perfil outro)
{
    if (outro == null)
        return false;
```

```

    bool interessesComuns =
        Interesses.Intersect(outro.Interesses).Any();

    bool mesmaLocal =
        string.Equals(Localizacao, outro.Localizacao,
            StringComparison.OrdinalIgnoreCase);

    return interessesComuns && mesmaLocal;
}
}
}

```

Nesse ponto, devemos criar o projeto de testes xUnit na mesma solução:

- No Gerenciador de Soluções, clique com o botão direito na solução (nível superior) (figura 21).
- Selecione Adicionar > Novo Projeto... (figura 21).
- Procure pelo template Projeto de Teste xUnit (C#) (figura 22).
- Nomeie o projeto como TinderApp.Tests (figura 23).
- Confirme.

O template de xUnit criado pelo Visual Studio normalmente já traz as referências básicas (xunit, xunit.runner.visualstudio e Microsoft.NET.Test.Sdk).

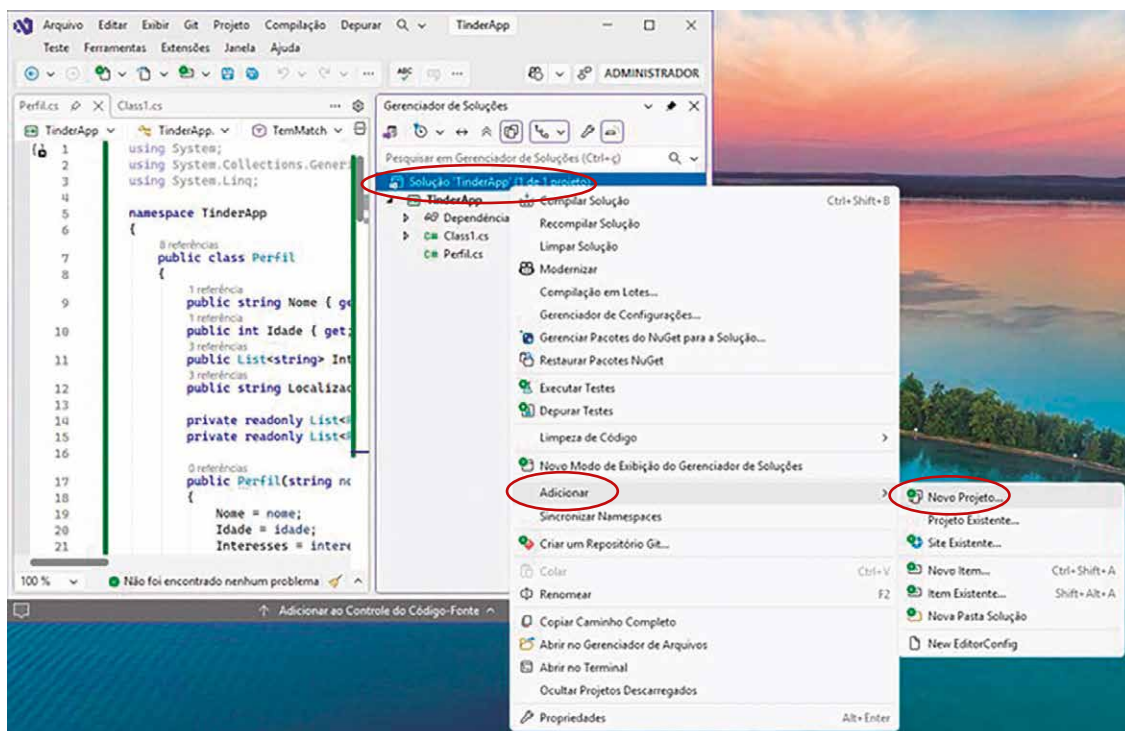


Figura 21 – Adicionando o projeto de teste na mesma solução

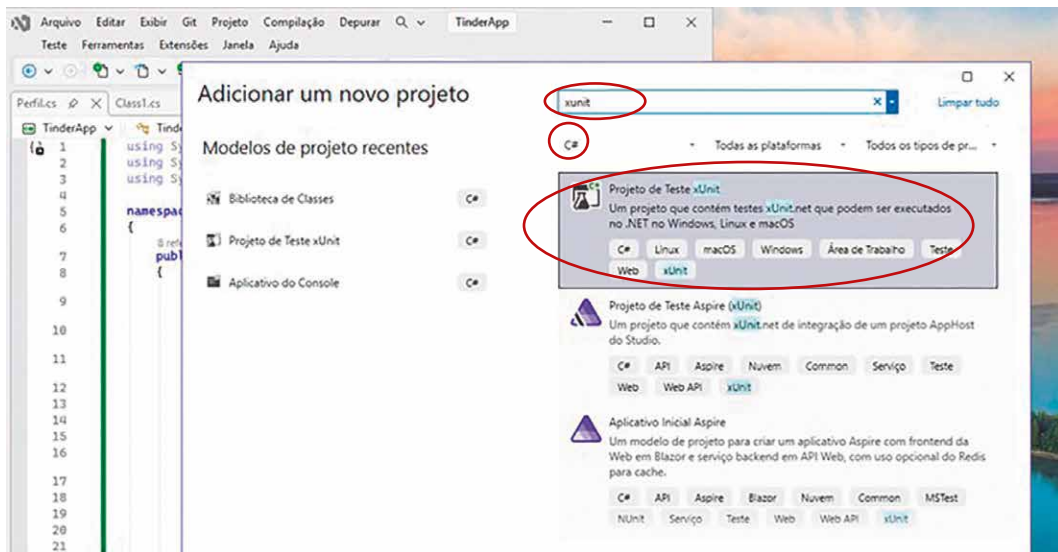


Figura 22 – Buscando e selecionando o template Projeto de Teste xUnit

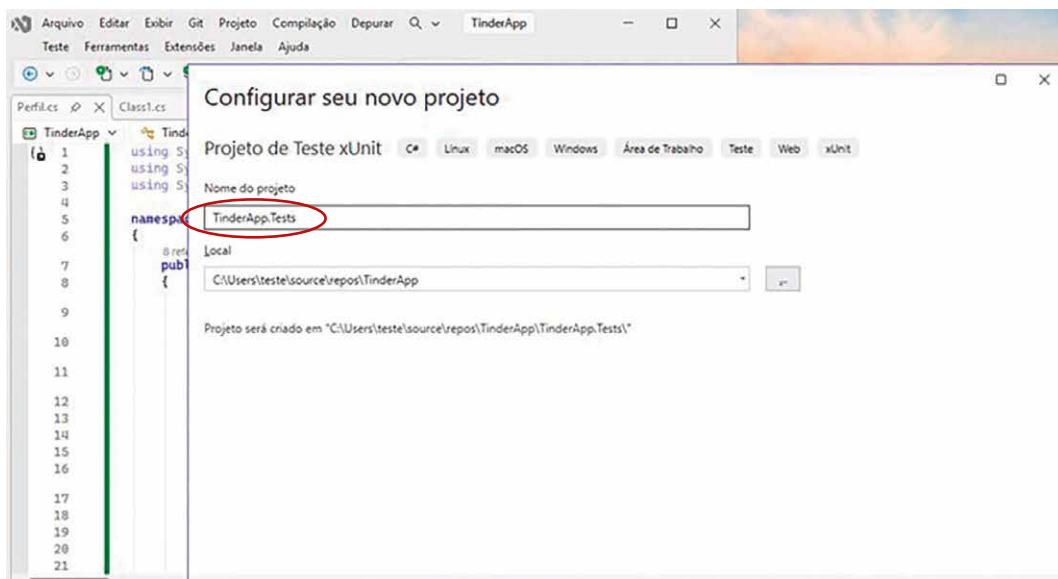


Figura 23 – Nomeando o projeto como TinderApp.Tests

Todavia, é importante verificar os pacotes NuGet do projeto de testes:

- No Gerenciador de Soluções, clique com o botão direito no projeto TinderApp.Tests (figura 24).
- Escolha Gerenciar Pacotes NuGet... (figura 24).
- Na aba Instalado (figura 25), verifique se estão presentes:
 - xunit
 - xunit.runner.visualstudio
 - Microsoft.NET.Test.Sdk

- Se algum estiver faltando, use a aba Procurar para instalá-los.

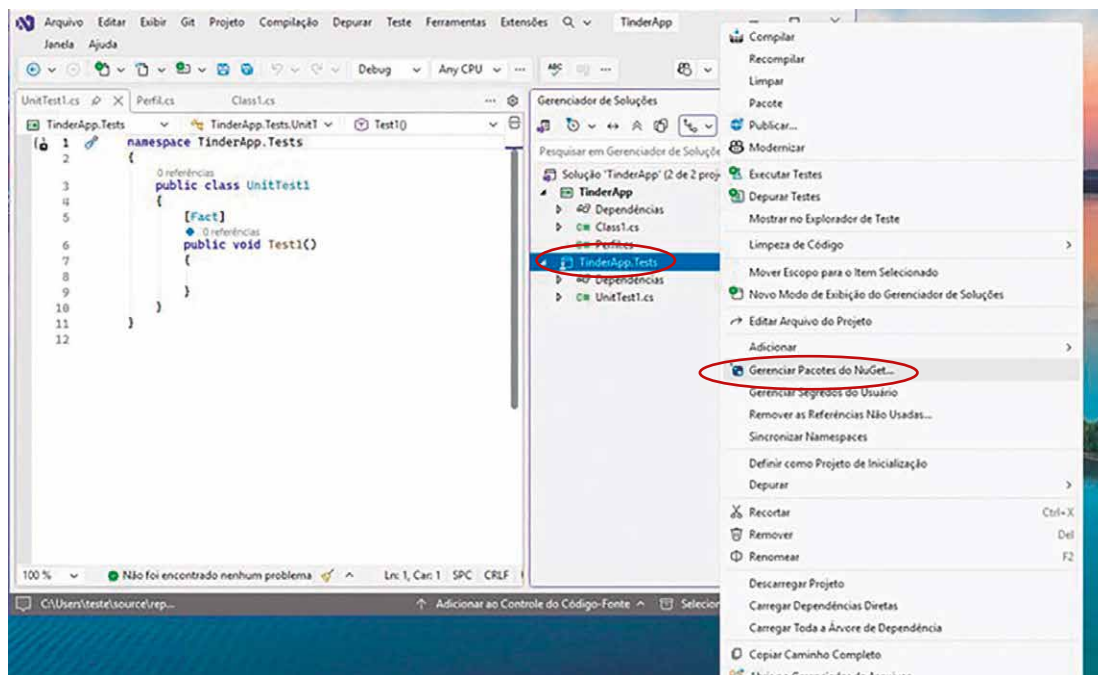


Figura 24 – Verificando os pacotes NuGet

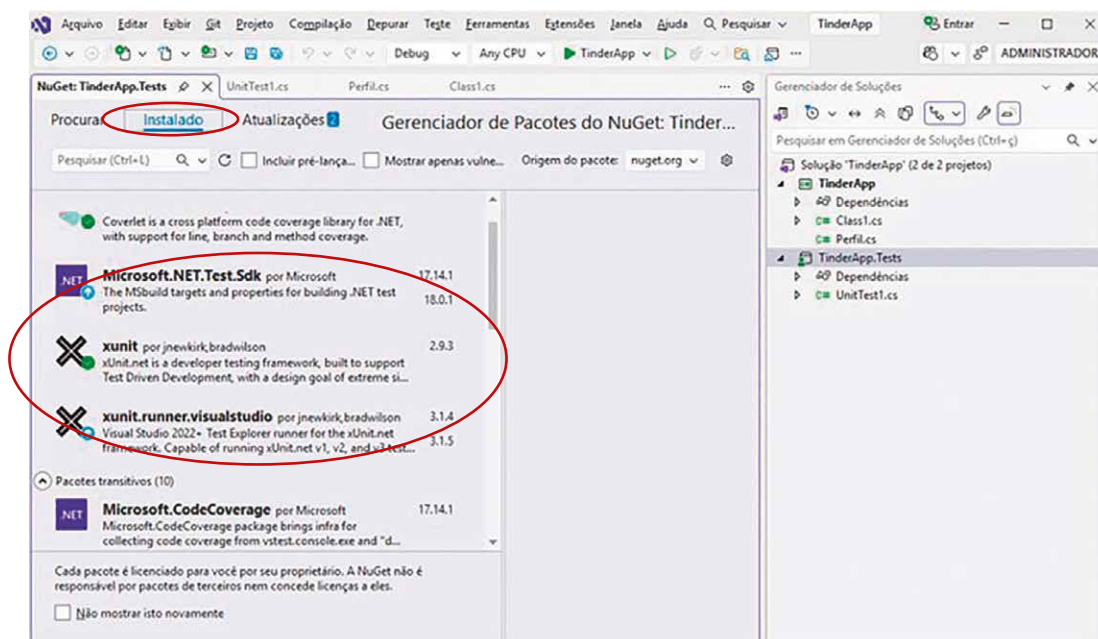


Figura 25 – Na aba Instalado, aparecem os pacotes na lista

Nossa próxima tarefa será adicionar uma referência ao projeto principal no projeto de testes:

- Ainda no Gerenciador de Soluções, clique com o botão direito no projeto TinderApp.Tests (figura 26).

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

- Selecione Adicionar > Referência de Projeto... (figura 26).
- Marque o projeto TinderApp (figura 27).
- Confirme.

Isso permite que os testes acessem a classe Perfil definida no projeto principal.

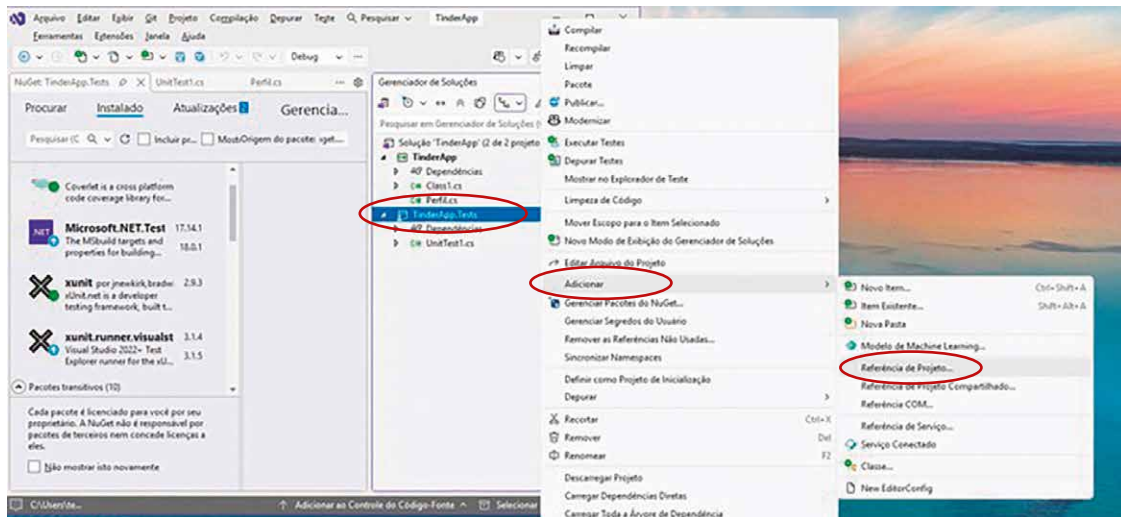


Figura 26 – Adicionando uma referência de projeto

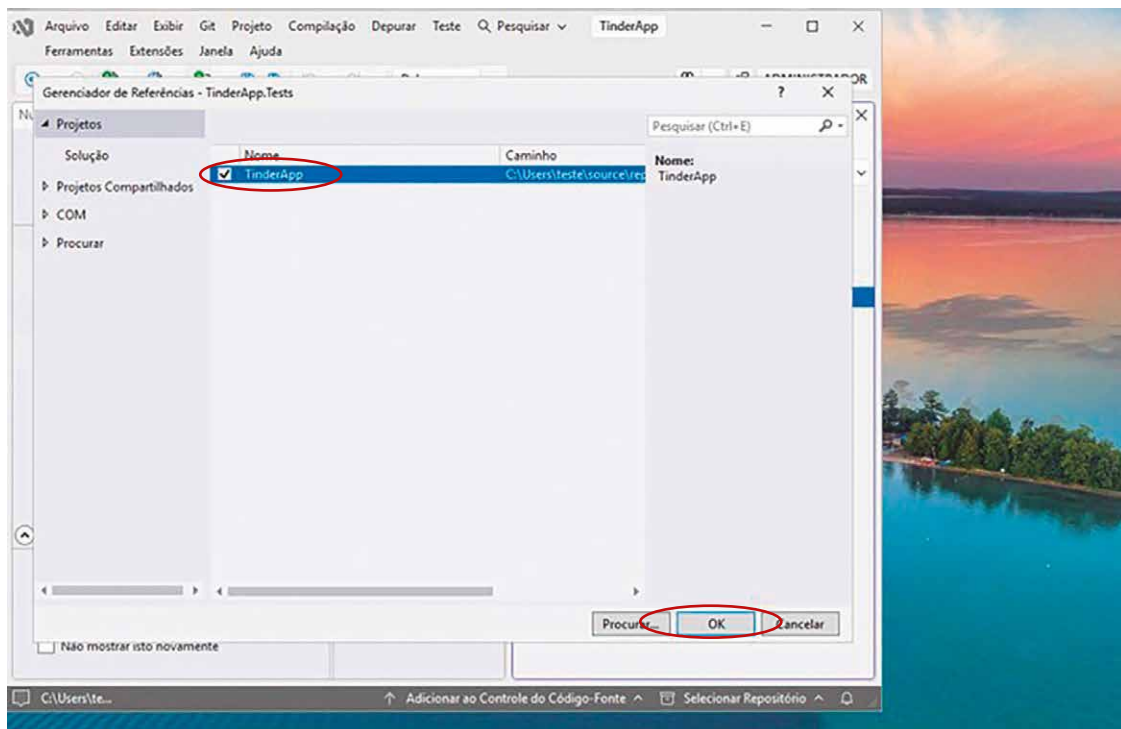


Figura 27 – Marcando o projeto TinderApp

Com isso, podemos escrever os testes automatizados em `TinderApp.Tests`. No projeto, adicione um novo arquivo de classe:

- Clique com o botão direito em `TinderApp.Tests` > Adicionar > Classe... (figura 28).
- Nomeie o arquivo como `TinderTests.cs` (figura 29).

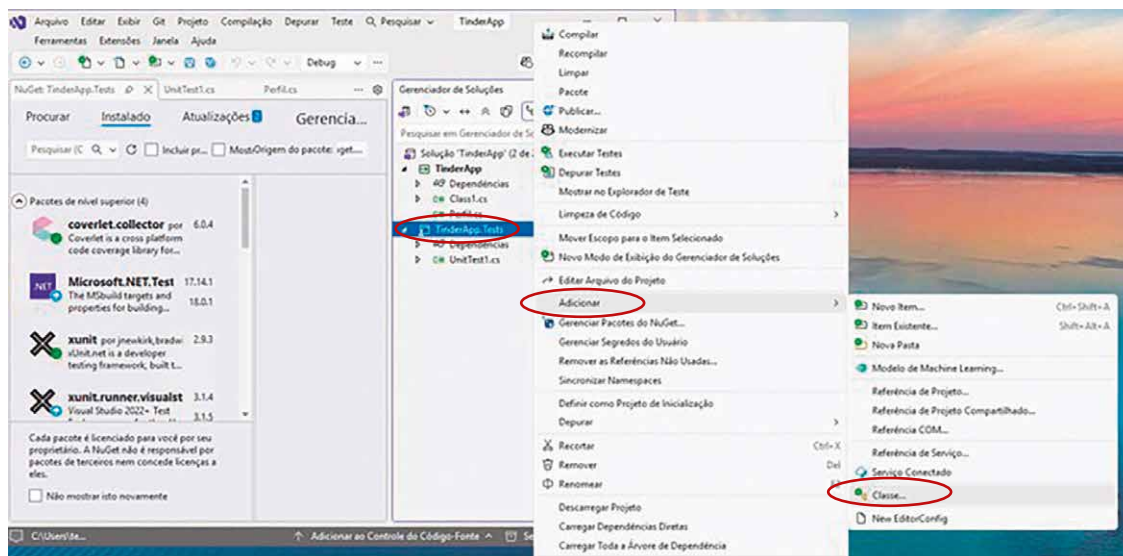


Figura 28 – Adicionando uma nova classe para escrever os testes

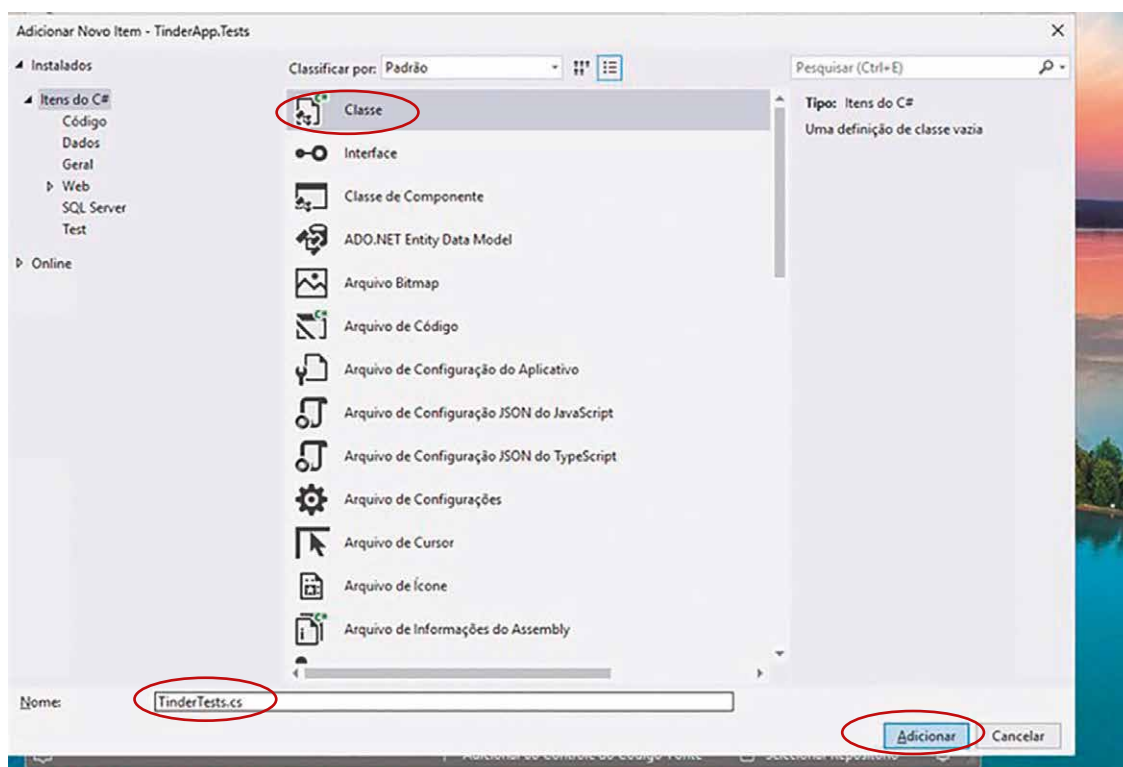


Figura 29 – Nomeando o arquivo de testes `TinderTests.cs`

Substitua o conteúdo gerado pelo código completo a seguir (figura 30).

```
using System.Collections.Generic;
using Xunit;
using TinderApp;

namespace TinderApp.Tests
{
    public class TinderTests
    {
        [Fact]
        public void MatchAconteceQuandoCurtidasSaoMutuas()
        {
            // Arrange
            var alice = new Perfil(
                "Alice",
                25,
                new List<string> { "música", "cinema" },
                "São Paulo");

            var bob = new Perfil(
                "Bob",
                30,
                new List<string> { "música", "esportes" },
                "São Paulo");

            // Act
            alice.Curtir(bob);
            bool houveMatch = bob.Curtir(alice); // agora há curtida mútua

            // Assert
            Assert.True(houveMatch);
            Assert.True(alice.TemMatchCom(bob));
            Assert.True(bob.TemMatchCom(alice));
        }

        [Fact]
        public void SemMatchQuandoApenasUmLadoCurte()
        {
            // Arrange
            var alice = new Perfil(
                "Alice",
                25,
                new List<string> { "música" },
                "Rio");

            var bob = new Perfil(
                "Bob",
                30,
                new List<string> { "esportes" },
                "Rio");

            // Act
            alice.Curtir(bob); // Bob não curte Alice
        }
    }
}
```

```

        // Assert
        Assert.False(alice.TemMatchCom(bob));
        Assert.False(bob.TemMatchCom(alice));
    }

    [Fact]
    public void CompatibilidadeExigeInteresseEmComumEMesmaLocalizacao()
    {
        // Arrange
        var maria = new Perfil(
            "Maria",
            28,
            new List<string> { "música", "viagem" },
            "Curitiba");

        var joao = new Perfil(
            "João",
            29,
            new List<string> { "viagem", "esportes" },
            "Curitiba");

        var carla = new Perfil(
            "Carla",
            27,
            new List<string> { "viagem" },
            "São Paulo");

        // Act / Assert
        // Maria e João: mesmo local e interesse em comum ("viagem")
        Assert.True(maria.EhCompatívelCom(joao));

        // Maria e Carla: interesse em comum, mas cidades diferentes
        Assert.False(maria.EhCompatívelCom(carla));
    }

    // Opcional: teste que falha de propósito, apenas se você quiser
    // demonstrar um teste com falha.
    // Se não quiser ver falha no Gerenciador de Testes, remova este método.
    [Fact]
    public void MatchErradoDeveFalhar_ExemploDidatico()
    {
        var alice = new Perfil(
            "Alice",
            25,
            new List<string> { "música" },
            "SP");

        var bob = new Perfil(
            "Bob",
            30,
            new List<string> { "dança" },
            "SP");

        alice.Curtir(bob);
        // Este Assert está errado de propósito: não há curtida mútua.
        Assert.True(alice.TemMatchCom(bob));
    }
}

```

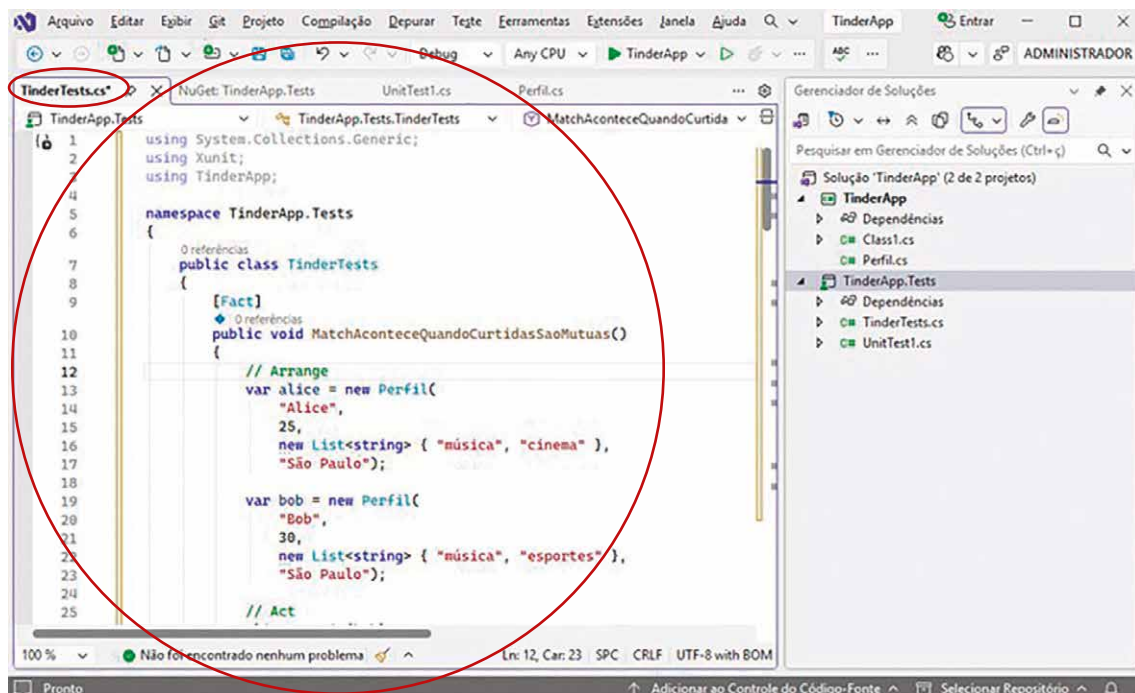



Figura 30 – Código de TinderTests.cs

Agora podemos executar os testes no Gerenciador de Testes:

- Compile a solução (Compilação > Compilar Solução) (figura 31).
- Abra o painel de testes: Menu Teste > Gerenciador de Testes (figura 32).
- No Gerenciador de Testes, os testes do projeto TinderApp.Tests devem aparecer listados (figura 33).
- Clique no ícone Executar Todos os Testes para executar todos os testes ou selecione testes individuais e use o menu de contexto (figura 34).
- Verifique o resultado. Os testes corretos devem aparecer como aprovados (figura 34).
- Note que o teste MatchErradoDeveFalhar_ExemploDidatico aparecerá como Falha, o que serve como demonstração visual de falha (figura 34).

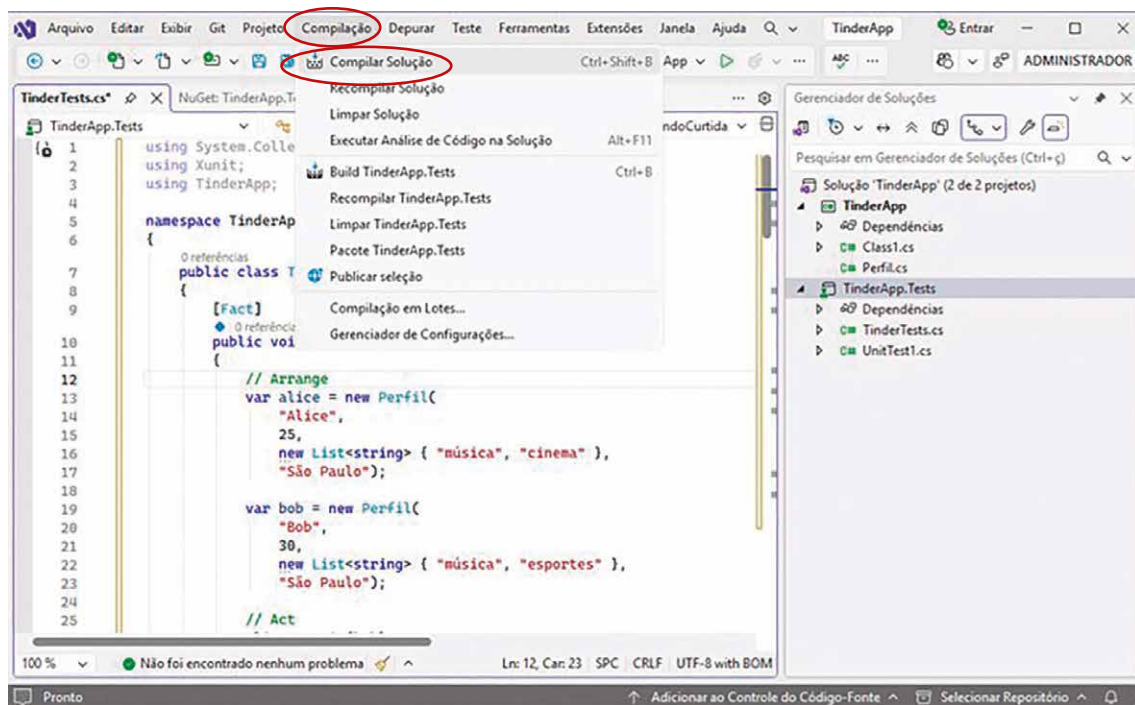


Figura 31 – Compilando a solução

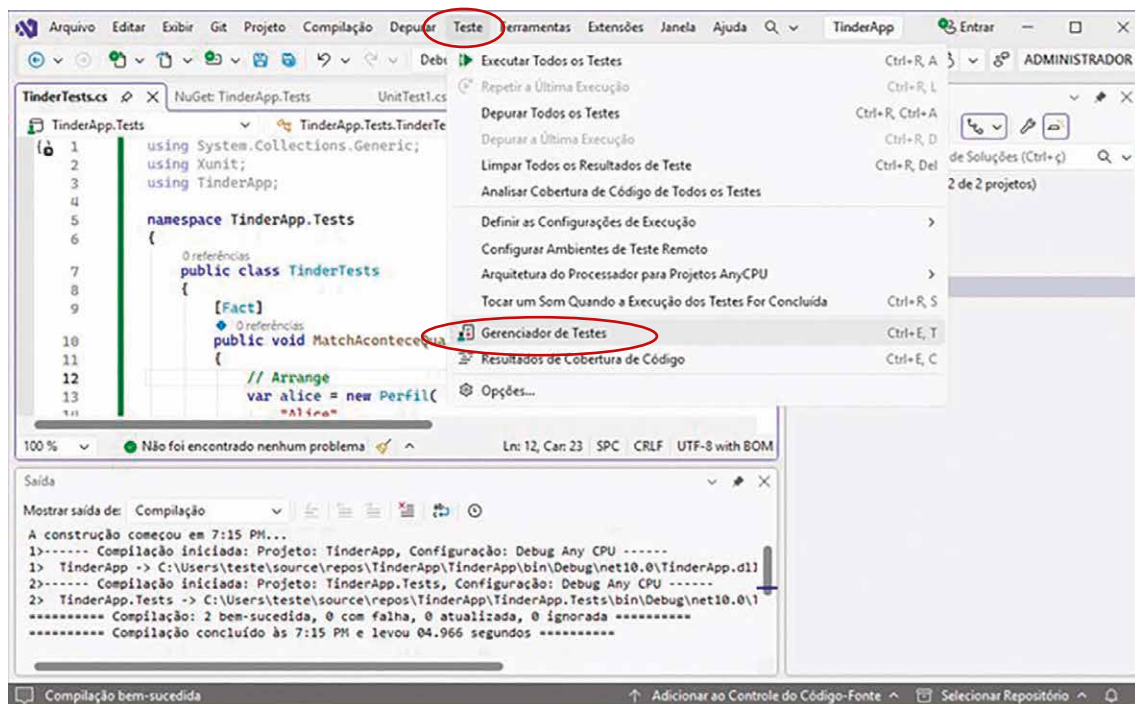


Figura 32 – Abrindo o Gerenciador de Testes

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

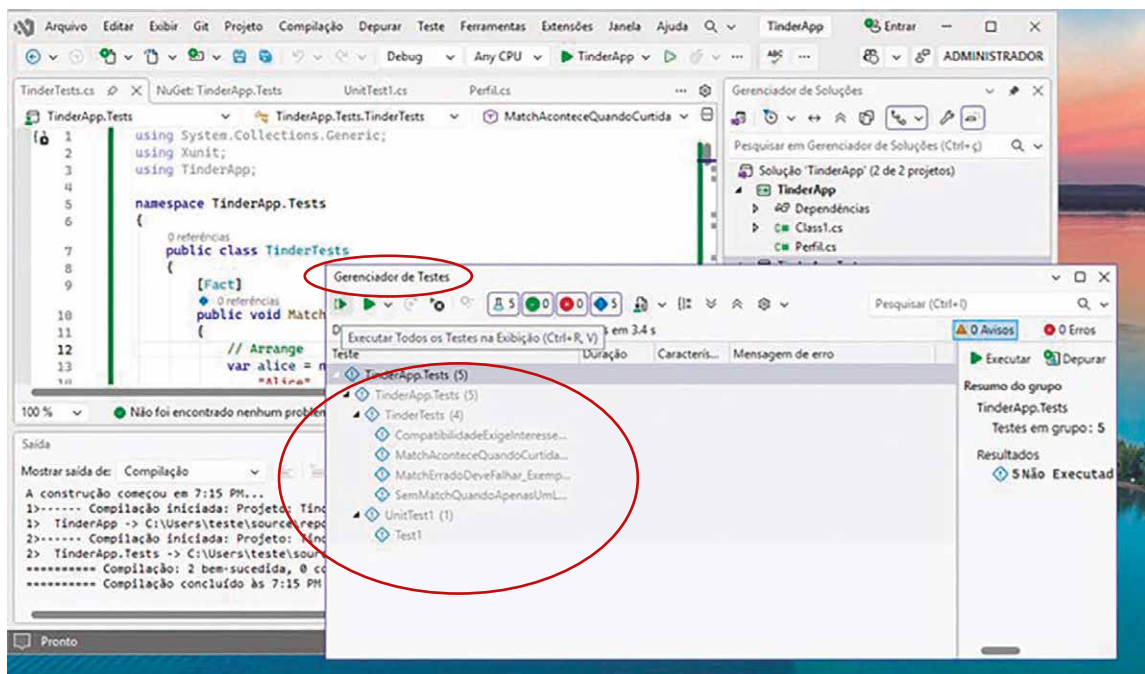


Figura 33 – Listando os testes disponíveis no Gerenciador de Testes

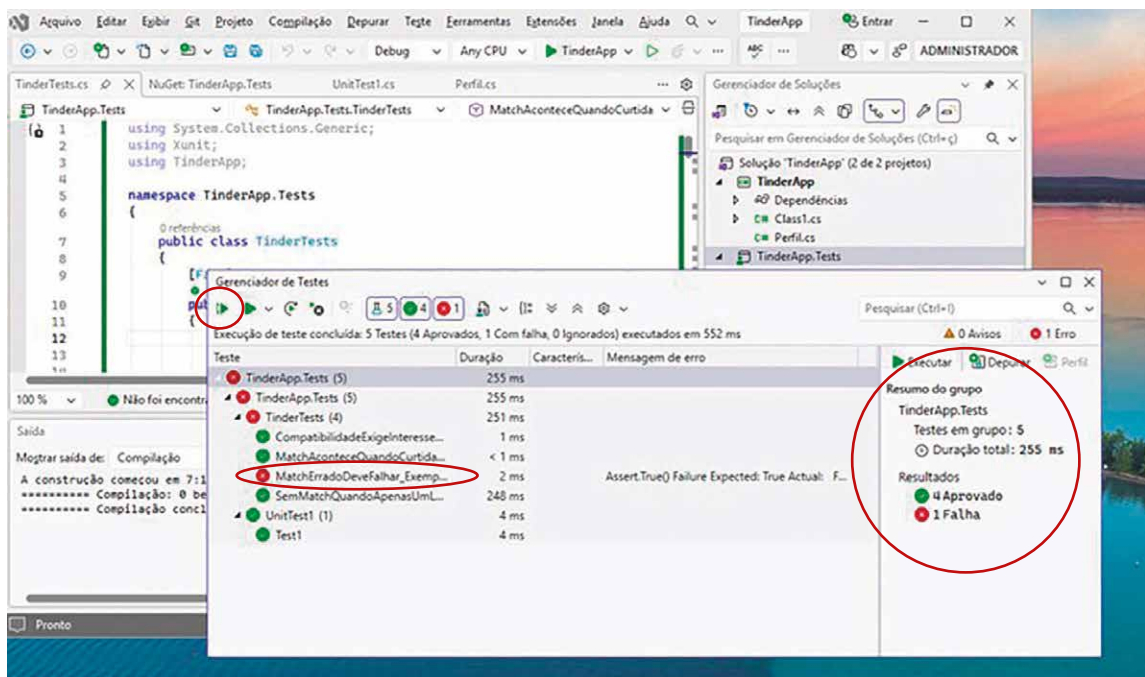


Figura 34 – Executando todos os testes

Nesse conjunto de testes que criamos, a falha esperada aconteceu justamente no teste **MatchErradoDeveFalhar_ExemploDidatico** (figura 34). A lógica da classe **Perfil** foi construída para registrar um match apenas quando há curtida mútua, isto é, quando cada perfil aparece ao mesmo tempo na lista de curtidos do outro.

Na implementação do método `Curtir`, a sequência é sempre a mesma: o perfil atual adiciona o outro em `perfisCurtidos` (caso ainda não esteja lá) e, em seguida, verifica se o outro perfil já tinha este perfil em sua própria lista de curtidos. Só quando essa condição de reciprocidade é verdadeira é que os dois perfis são colocados nas listas de matches um do outro e o método retorna `true`. Se a reciprocidade não existe, nenhuma lista de matches é atualizada e o método retorna `false`.

No teste, criamos dois perfis, `alice` e `bob`, e chamamos apenas `alice.Curtir(bob)`. Isso faz com que `bob` apareça na lista de curtidos de `alice`, mas em nenhum momento há uma chamada correspondente `bob.Curtir(alice)`. Quando o método `Curtir` é executado para `Alice`, ele verifica se `bob.perfisCurtidos` contém `alice`.

Como `Bob` nunca curtiu `Alice`, essa lista não contém o perfil de `Alice`, a condição de curtida mútua não é satisfeita, o bloco que adiciona cada um na lista de matches do outro não é executado e o retorno é `false`. Além disso, a lista interna de matches de `Alice` continua vazia em relação a `Bob`.

O problema do teste está exatamente na expectativa configurada com `Assert.True(alice.TemMatchCom(bob))`. O método `TemMatchCom` simplesmente verifica se `Bob` está presente na lista de matches de `Alice`. Como o match nunca foi registrado, `TemMatchCom(bob)` devolve `false`.

O `xUnit` compara esse resultado com o que o teste declara como esperado: ele espera `true`, mas recebe `false`. Essa discrepância entre resultado real e valor esperado é o que faz o teste ser marcado como `Falha` no Gerenciador de Testes.

Do ponto de vista da lógica de negócio, o que está errado não é o comportamento da classe `Perfil`, e sim a suposição escrita no teste: o teste afirma que deveria existir match após apenas uma curtida, enquanto a regra implementada exige reciprocidade.

Se você executar os outros testes corretos, como o que primeiro chama `alice.Curtir(bob)` e depois `bob.Curtir(alice)`, perceberá que eles passam sem erro porque refletem exatamente a lógica interna da classe: primeiro um lado curte, depois o outro, as listas `perfisCurtidos` ficam simétricas, o bloco de match é acionado e os dois perfis passam a ser considerados compatíveis nesse sentido.

Já o teste didático que gera `Falha` é útil justamente para demonstrar, na prática, que o framework de testes detecta qualquer divergência entre o comportamento real do código e as expectativas programadas nos asserts.

A adoção de testes automatizados com `xUnit` traz benefícios tangíveis: detecta regressões cedo, documenta o comportamento esperado do código e facilita refatorações. Os testes bem escritos tornam-se documentação executável e ajudam a manter o código saudável ao longo do tempo.

No contexto acadêmico e industrial atual, conhecer ferramentas como `xUnit` e Gerenciador de Testes é fundamental para garantir a confiabilidade de aplicações em `C#`. O desenvolvedor moderno deve aproveitar esses recursos para implementar desenvolvimento orientado a testes (TDD) e `continuous integration/continuous delivery (CI/CD)` eficazes, sobretudo porque frameworks modernos, como o `xUnit`, acompanham novas versões da linguagem.

8 MEMÓRIA E CONCORRÊNCIA INTRODUTÓRIA

Desenvolvedores que trabalham com C# muitas vezes têm a sensação de que o ambiente gerenciado resolve automaticamente os problemas de memória e concorrência, graças ao GC e às abstrações de alto nível oferecidas pelo .NET.

Essa impressão, porém, costuma ruir conforme a aplicação cresce e começam a surgir pausas perceptíveis em tempo de execução, consumo excessivo de memória e travamentos difíceis de reproduzir. A ausência de ponteiros explícitos não elimina as decisões delicadas sobre como alocar e estruturar os dados, nem dispensa o cuidado com a forma como o código dispara operações paralelas e assíncronas. Entender o que o runtime está de fato fazendo com objetos, threads e tarefas deixa de ser curiosidade técnica e passa a ser condição para manter aplicações responsivas, previsíveis e sustentáveis a longo prazo.

Nesse contexto, a distinção entre tipos por valor e tipos por referência, frequentemente apresentada como detalhe de sintaxe em cursos introdutórios, adquire importância prática concreta. A decisão de representar um conceito de domínio como struct ou como class afeta diretamente o padrão de alocação (pilha, heap gerenciado, boxing em coleções genéricas), o custo das cópias, a interação com o GC e até a clareza dos invariantes daquele tipo.

Escolhas ingênuas podem levar a estruturas que geram pressão desnecessária sobre o GC ou dificultam o raciocínio sobre quem possui determinado estado. Ao mesmo tempo, aproveitar struct para dados pequenos, com semântica de valor bem definida, ajuda a reduzir alocações e tornar o comportamento do programa mais previsível.

A reflexão sobre esses elementos mostra que projeto de tipos e gerenciamento de memória não estão separados: caminham juntos e impactam tanto desempenho quanto legibilidade. A discussão se torna ainda mais sensível ao entrar no território da concorrência. Em sistemas operacionais modernos, processos e threads formam a base da execução, e cada thread representa uma linha de controle que o escalonador pode alternar entre núcleos de CPU.

Trabalhar diretamente com threads em C# oferece grande poder, mas também expõe o desenvolvedor a problemas clássicos de concorrência: condições de corrida, deadlocks, uso inadequado de recursos e dificuldade em raciocinar sobre o fluxo de execução quando múltiplas operações avançam em paralelo.

Surge, então, a necessidade de modelos mais abstratos, nos quais o foco esteja na unidade de trabalho a ser executada, e não na mecânica de criação e sincronização de threads individuais. É nesse cenário que o ecossistema .NET passa a privilegiar Tasks e bibliotecas que escondem os detalhes de agendamento, sem eliminar a responsabilidade de compreender seus fundamentos.

A camada de programação assíncrona, construída em torno de Task, async/await e cancelamento com CancellationToken, responde justamente às dificuldades de conciliar desempenho com código legível.

Muitas aplicações modernas são dominadas por operações I/O-bound, nas quais o gargalo não está no cálculo local, mas na espera por respostas de rede, disco ou outros serviços externos; se essas esperas forem tratadas de modo bloqueante, a interface gráfica congela, servidores ficam incapazes de atender a novas requisições e a percepção de qualidade do software cai rapidamente.

O modelo baseado em Tasks permite expressar essas operações como trabalhos independentes, passíveis de composição, enquanto a sintaxe `async/await` oferece uma forma mais direta de escrever fluxos assíncronos. A introdução do cancelamento cooperativo, por sua vez, responde à necessidade de interromper operações longas sem deixar recursos pendurados e sem recorrer a mecanismos abruptos.

Assim, este tópico se insere em um conjunto de preocupações que vão da alocação de memória ao controle refinado de concorrência, fornecendo ao leitor o pano de fundo conceitual necessário para compreender por que essas construções de C# são centrais no desenvolvimento profissional.

8.1 Valor x referência, GC, struct x class

A linguagem C# executa sobre o ambiente gerenciado do .NET, que provê gerenciamento automático de memória por meio de um coletor de lixo (garbage collector, GC). Em vez de liberar memória manualmente, como nas linguagens C/C++, o CLR (Common Language Runtime) aloca objetos em um heap gerenciado e os descarta quando não são mais acessíveis.

Cada chamada a `new` calcula o tamanho dos campos do objeto, adiciona o overhead (por exemplo, ponteiro de tipo e índice de bloco de sincronização) e então, se houver espaço suficiente na região atual do heap, aloca o objeto, zera a memória, invoca o construtor e ajusta o ponteiro `NextObjPtr` para o próximo espaço livre.

Esse processo sequencial de alocação é extremamente rápido, beneficiando-se da localidade de referência: objetos relacionados tendem a ser alocados próximos no heap, reduzindo faltas de cache e tornando o acesso eficiente. Contudo, como a memória não é infinita, o CLR emprega o GC para deletar objetos inacessíveis.

Diferentemente de sistemas que usam contagem de referências, o GC do .NET é um coletor por rastreamento: suspende as threads, zera bits de marcação em todos os objetos e então marca recursivamente todos os objetos alcançáveis a partir das raízes (referências ativas).

Objetos não marcados são considerados inalcançáveis e podem ser removidos. Na fase de compactação, os objetos sobreviventes são movidos juntos para restaurar a localidade e manter o espaço livre contíguo, melhorando o desempenho geral. Esse esquema evita problemas de ciclos que existiriam em contagem de referências tradicional: o CLR concentra-se **apenas em variáveis de tipos por referência**, já que somente elas podem apontar para objetos no heap; variáveis de tipos por valor contêm a instância diretamente e não são consideradas raízes.

Em resumo, a alocação de objetos em C# é transparente e eficiente, e o GC cuida automaticamente da coleta de memória não usada, prevenindo vazamentos e corrupção de memória, comuns em linguagens de gerenciamento manual.

Como vimos anteriormente, tipos por valor e tipos por referência têm comportamentos distintos. Variáveis de tipos por valor (por exemplo, `int`, `struct` definido pelo usuário) armazenam diretamente os dados da instância; na prática, são geralmente alocadas na pilha (`stack`) ou inline dentro de objetos. Já variáveis de tipos por referência (por exemplo, instâncias de `class`) armazenam apenas uma referência (ponteiro) para um objeto no `heap`.

Isso implica que, ao atribuir uma variável de tipo por valor, ocorre uma cópia independente, ao passo que, ao atribuir uma variável de tipo por referência, copia-se apenas a referência, ambos os nomes referindo o mesmo objeto.



Lembrete

Uma classe é um tipo por referência; uma `struct` é um tipo por valor e, conseqüentemente, `structs` representam tipos simples, com semântica de valor, nos quais a atribuição copia efetivamente o valor.

Considere o seguinte código ilustrativo:

```
int a = 10;
int b = a;
a = 20;
// a vale 20, b continua 10 (tipos por valor).
class Pessoa { public int Id; }
Pessoa p1 = new Pessoa { Id = 1 };
Pessoa p2 = p1;
p1.Id = 2;
// p1.Id e p2.Id agora valem 2, pois p1 e p2 referem o mesmo objeto.
```

Essa diferença de semântica leva a distintos padrões de uso: `structs` são preferencialmente usados para tipos imutáveis ou pequenos, já que evitam sobrecarga de alocação no `heap` e coleta de lixo. Classes, por outro lado, suportam herança e são usadas para objetos mais complexos.

Do ponto de vista de memória, isso significa que objetos instanciados de classes são garantidamente alocados no `heap` (sujeitos ao GC), ao passo que `structs` podem ficar na pilha ou dentro de outros objetos. Mesmo `structs` podem eventualmente ir no `heap` se forem submetidos a `boxing` ou se fizerem parte de um array ou de um campo de classe. Além disso, `structs` não podem ter construtores sem parâmetro definido pelo usuário e são implicitamente selados.

Essas características influenciam o projeto de tipos em C#. Por exemplo, é comum usar `structs` para representar pontos, retângulos, datas e pequenos dados com comportamento de valor, enquanto classes encapsulam a lógica de negócio e os estados mais complexos.

8.2 Task, async/await, cancelamento com CancellationToken

A programação concorrente em C# utiliza rotinas de alta abstração chamadas `Tasks` (tarefas), as quais representam operações assíncronas potencialmente em execução paralela.

Em vez de manipular diretamente threads, o .NET introduziu a Task Parallel Library (TPL), na qual cada `Task` corresponde a uma unidade de trabalho distribuída pelo **thread pool**. Essa estratégia torna a programação concorrente mais segura e legível, pois a tarefa abstrai a criação e a sincronização de **threads**.

Threads, no contexto de sistemas operacionais e de C#, são a linha de execução que de fato consome CPU dentro de um processo. Em termos clássicos, Silberschatz, Galvin e Gagne (2018) definem thread como a unidade básica de utilização da CPU: ela possui um identificador próprio, um contador de programa, um conjunto de registradores e uma pilha, compartilhando com outras threads do mesmo processo o código, os dados e demais recursos do sistema operacional, como arquivos abertos e sinais.

Isso significa que, dentro de um único processo, é possível ter várias threads executando em paralelo (realmente paralelas em máquinas multicore, ou intercaladas pelo escalonador em máquinas de um único núcleo), cada uma com seu fluxo de instruções, embora todas enxerguem o mesmo espaço de endereçamento do processo.



Saiba mais

Para se aprofundar em threads no nível de sistemas operacionais e na forma como elas aparecem no ecossistema .NET e em C#, duas referências adequadas são:

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating system concepts*. 10. ed. Hoboken: John Wiley & Sons, 2018.

CLEARY, S. *Concurrency in C# cookbook: asynchronous, parallel, and multithreaded programming*. 2. ed. Sebastopol: O'Reilly Media, 2019.

O primeiro livro apresenta a definição clássica de threads e o contexto de processos e escalonamento em sistemas operacionais. O segundo discute, em termos práticos, como o .NET utiliza o thread pool e a TPL e por que, na maior parte das aplicações C#, faz mais sentido pensar em `Tasks` e deixar o gerenciamento de threads para o runtime.

Essa separação entre processo e thread é importante para entender as `Tasks` em C#. O **processo** é o contêiner pesado que define o espaço de endereçamento, os handles de arquivo, as conexões de rede e demais recursos globais. As threads são as entidades leves que o sistema operacional agenda para executar instruções nesse espaço.

Como cada thread tem a própria pilha e registradores, duas threads podem estar executando trechos diferentes do mesmo programa ao mesmo tempo, compartilhando a mesma memória. Essa combinação torna a criação de threads muito mais barata do que a criação de novos processos, motivo pelo qual a literatura de sistemas operacionais trata threads como lightweight processes.

Em .NET e C#, essa ideia é preservada, mas aparece com uma camada de abstração adicional. O CLR expõe threads gerenciadas, representadas pela classe `System.Threading.Thread`, que normalmente mapeiam para threads do sistema operacional. Na prática, porém, o estilo moderno de programação concorrente recomendado em C# não é criar e controlar thread manualmente, e sim trabalhar com a TPL e com o tipo `Task`.

A infraestrutura de concorrência em C# se apoia em um thread pool mantido pelo runtime: esse thread pool mantém um conjunto de threads trabalhadoras (worker threads) que ficam esperando trabalho; a própria biblioteca gerencia quantas threads existem no pool e distribui as unidades de trabalho entre elas.

Em outras palavras, as threads continuam existindo, mas o desenvolvedor lida principalmente com `Tasks`, que são objetos de mais alto nível representando operações assíncronas, enquanto a criação, a reutilização e o escalonamento das threads físicas são tratados pelo thread pool do .NET.

A thread é a entidade de baixo nível que o sistema operacional conhece e agenda na CPU. A `Task` é uma abstração introduzida pela TPL para representar um trabalho, uma computação que pode estar em andamento, concluída, cancelada ou falha, com suporte integrado a composição (continuations), cancelamento cooperativo e integração com o `async/await`.

A TPL pega essas `Tasks` e as executa em cima do thread pool, ou seja, em cima de threads reaproveitadas, evitando o custo de criar e destruir threads frequentemente, algo que a literatura de concorrência em C# enfatiza como ponto central de desempenho e simplicidade. Assim, pode-se dizer que, conceitualmente, threads continuam sendo o motor de execução, enquanto `Tasks` são a unidade de trabalho que você manipula na maior parte do código C# moderno.

Ao iniciar uma `Task`, por exemplo, com `Task.Run(() => /* trabalho */)`, o código é executado assincronamente. O programador pode em seguida aguardar seu término com `await` ou combinar tarefas com métodos como `Task.WhenAll` ou `Task.WhenAny`.

O padrão Task-Based Asynchronous Pattern (TAP) foi formalizado com o C# 5.0, permitindo escrever código assíncrono em estilo imperativo; observa-se que operações assíncronas estão disponíveis por meio do padrão especial `async/await`, simplificando, desse modo, a programação assíncrona. Em vista disso, usar `async` e `await` para operações assíncronas confere ao código o comportamento de aguardar tarefas sem bloquear a thread chamadora.

Internamente, um método assíncrono definido com o modificador `async` e que retorna `Task` ou `Task<T>` é compilado em uma máquina de estados. Quando esse método encontra um `await`

em uma `Task` ainda não completada, ele registra uma continuação para retomar o fluxo quando a operação terminar.

Assim, o programa aparenta saltar para outras instruções (como em um callback) enquanto aguarda o fim da tarefa; esse comportamento transparente decorre de o compilador C# transformar o código assíncrono em código baseado em tarefas. Um método simples que soma dois números pode ser visto no exemplo a seguir, que ilustra a sintaxe assíncrona:

```
async Task<int> SomaAsync(int a, int b) {  
    // Executa o cálculo de soma numa tarefa paralela sem bloquear a thread  
    atual  
    return await Task.Run(() => a + b);  
}
```

Quando `await SomaAsync(3,4)` é executado, o método chamador inicia a operação assíncrona e devolve o controle ao ambiente de chamada (por exemplo, ao loop de mensagens da interface gráfica) até que a `Task` seja concluída. Quando a `Task` termina com sucesso, o fluxo retoma naquele ponto, agora com o resultado 7. Esse modelo permite escrever código com aparência sequencial, mas sem bloquear a thread da interface, o que é essencial tanto em operações intensivas em CPU quanto, principalmente, nas operações I/O-bound.

Diz-se que uma operação é I/O-bound quando o tempo total de execução é dominado pela espera por entrada/saída (rede, disco, sistema de arquivos, soquete etc.) e não pelo processamento de CPU. Em uma chamada HTTP, por exemplo, a maior parte do tempo é gasta aguardando o servidor responder e os dados viajarem pela rede. Se essa espera for feita de forma síncrona, a thread fica bloqueada, sem poder processar outras mensagens da interface ou outras requisições.

Em cenários práticos, muitas operações de I/O (como chamadas web, acesso a banco ou leitura de arquivos) expõem APIs assíncronas nativas que retornam diretamente `Task` ou `Task<T>`. O método `HttpClient.GetStringAsync(uri)`, por exemplo, inicia uma requisição HTTP GET para o endereço indicado e devolve imediatamente uma `Task<string>`.

Essa `Task<string>` representa uma operação em andamento que, no futuro, produzirá uma string com o corpo da resposta. O fato de o método retornar uma `Task` é crucial: isso permite que o chamador use `await` sem bloquear a thread.

Enquanto a resposta ainda não chegou, o runtime pode liberar a thread atual para atender a outras tarefas; quando os dados chegam e a tarefa é marcada como concluída, a continuação do método é reagendada, e a variável que recebe o `await` passa a conter o texto retornado pelo servidor. Desse modo, a aplicação permanece responsiva mesmo em presença de operações I/O-bound potencialmente demoradas, mantendo um estilo de código direto e legível.

Métodos marcados com `async` e com retorno `Task` permitem ao programador **iniciar tarefas de forma não bloqueante**, ao passo que o uso de `await` imediatamente suspende o método até a

conclusão. A seguir, temos um trecho comum em aplicações .NET, que inicia uma tarefa longa e depois aguarda sem travar a aplicação:

```
public async Task CalcularOperacaoDemoradaAsync() {  
    // Inicia uma tarefa que simula operação custosa  
    await Task.Delay(2000); // Espera 2 segundos de forma assíncrona  
    Console.WriteLine("Operação demorada concluída");  
}
```

Assim que se chama `await CalcularOperacaoDemoradaAsync()`, o fluxo principal do programa retorna a outros trabalhos até que os 2.000 ms sejam completados. Para executar várias tarefas em paralelo, podemos usar métodos como `Task.WhenAll`, que recebe uma lista de `Task` e retorna uma nova `Task` completada quando todas terminarem.

```
Task<int> t1 = Task.Run(() => ObterDado1());  
Task<int> t2 = Task.Run(() => ObterDado2());  
int[] resultados = await Task.WhenAll(t1, t2);  
// "resultados" contém [resultado de ObterDado1, resultado de ObterDado2]
```

Esse recurso permite paralelizar operações independentes sem sincronismo manual. Caso precise reagir à primeira tarefa concluída, `Task.WhenAny` ou `await Task.WhenAny(t1, t2)` retornam a tarefa que finalizou primeiro. Cabe ainda salientar que exceções em `Tasks` se propagam quando se aguarda (`await` ou `Task.WaitAll`), sendo tratadas por blocos `try/catch` usuais.

O modelo de `Tasks` facilita expressar fluxos complexos de concorrência sem recorrer a callbacks clássicos. Porém, para uma programação correta, é necessário cooperar no controle de cancelamento e sincronização, tema que abordaremos a seguir.

Em programação assíncrona profissional, é imprescindível permitir que operações longas sejam canceladas de forma cooperativa, seja por tolerância a tempo-limite ou pela ação do usuário. O .NET disponibiliza a classe `CancellationToken` como mecanismo padrão para esse fim.

Basicamente, o chamador cria um `CancellationTokenSource` e passa seu `Token` (uma struct) aos métodos assíncronos que suportam cancelamento. Esses métodos consultam periodicamente a propriedade `ct.IsCancellationRequested` ou usam sobrecargas de métodos assíncronos, como `GetAsync(url, ct)`, que cancelam automaticamente quando o token é sinalizado.

Operações assíncronas podem ser canceladas da mesma forma que tarefas paralelas, passando um `CancellationToken` e chamando o método `Cancel()` no `CancellationTokenSource`. Em outras palavras, ao detectar a necessidade de parar, chama-se `cts.Cancel()`; no código chamador. Isso faz com que qualquer método que esteja aguardando aquele token lance uma exceção `OperationCanceledException` ou termine normalmente, de modo a liberar recursos.

Para habilitar um cancelamento cooperativo limpo, é prática comum utilizar `ct.ThrowIfCancellationRequested()` em pontos apropriados para interromper a execução do método quando

solicitado. Um exemplo ilustrativo de uso de `CancellationToken` em um método assíncrono de longo prazo seria:

```
public async Task<int> ObterContagemPalavrasAsync(CancellationToken ct) {
    // Simula delay para permitir cancelamento
    await Task.Delay(1000, ct);
    // Exemplo de chamada externa suportando cancelamento
    HttpResponseMessage resp = await httpClient.GetAsync(urlArquivo, ct);
    string texto = await resp.Content.ReadAsStringAsync(ct);
    string[] palavras = texto.Split(new[] { ' ', '\n', '\r' }, StringSplitOptions.
RemoveEmptyEntries);
    ct.ThrowIfCancellationRequested(); // Verifica cancelamento antes de retornar
    return palavras.Length;
}
```

Nesse trecho, `GetAsync` e `ReadAsStringAsync` recebem o token `ct`, e um breve `Task.Delay` inicial usa o token como demonstração de cancelamento antecipado. Caso `cts.Cancel()` seja chamado antes da conclusão, o método lançará `OperationCanceledException`, que pode ser capturada no chamador (geralmente um bloco `try/catch`) para tratar o cancelamento de modo gracioso. O código que inicia a chamada protegeria `ObterContagemPalavrasAsync` assim:

```
var cts = new CancellationTokenSource();
try {
    Task<int> tarefa = ObterContagemPalavrasAsync(cts.Token);
    // ... talvez em outra thread, após algum evento:
    // cts.Cancel(); // sinaliza cancelamento
    int total = await tarefa;
    Console.WriteLine($"Palavras: {total}");
}
catch (OperationCanceledException) {
    Console.WriteLine("Operação cancelada pelo usuário.");
}
```

Esse padrão cooperativo segue um fluxo bem definido: o chamador cria o `CancellationTokenSource`, chama o método assíncrono passando `cts.Token` e, quando necessário, invoca `cts.Cancel()`. O método assíncrono, por sua vez, respeita esse token em pontos adequados, lançando exceção para abortar. Segundo literaturas de referência, esse fluxo de cancelamento é comum e recomendado nas aplicações .NET.

Também vale mencionar que é possível encadear tokens (por exemplo, criar um novo token que combine cancelamentos de múltiplas fontes) usando `CancellationTokenSource.CreateLinkedTokenSource`, o que assegura que qualquer solicitação seja propagada adequadamente. Em geral, todo método público de longa duração deve aceitar opcionalmente um `CancellationToken` para oferecer controle ao consumidor do método.

Em resumo, o modelo task-based de programação assíncrona em C# (TAP) fornece uma base rica para lidar com concorrência e I/O assíncrono de forma expressiva. Tarefas (`Task` e `Task<T>`) representam

unidades de trabalho que podem ser lançadas ao thread pool ou a provedores de I/O assíncrono, e seu resultado pode ser obtido com `await` sem bloquear a thread de chamada.

O uso de `async/await` simplifica a programação assíncrona, mantendo o código linear e fácil de entender. Além disso, práticas de cancelamento cooperativo (usando `CancellationToken`) permitem que aplicativos permaneçam responsivos e graciosos diante de solicitações de cancelamento, conforme recomendado por Cleary (2019), que discute amplamente o uso de `Tasks` e cancelamento cooperativo em C#.

Em conjunto, essas técnicas (tarefas, `async/await` e cancelamento cooperativo) formam o alicerce de um desenvolvimento moderno em C# para aplicativos reativos e escaláveis e devem ser bem compreendidas em cenários práticos para garantir desempenho e robustez no código paralelo.



Resumo

Esta unidade apresentou uma visão organizada sobre falhas e qualidade em C# ao distinguir três camadas de problemas: os erros detectáveis estaticamente pelo compilador, as falhas que emergem em tempo de execução e os erros de lógica que produzem resultados incorretos sem sinais técnicos evidentes.

Erros de compilação decorrem da violação de regras da linguagem e do sistema de tipos e impedem a geração do executável, funcionando como uma primeira barreira de segurança. Uma vez superada essa etapa, o foco se desloca para as exceções do .NET, modeladas como objetos derivados de `System.Exception`, que sinalizam condições anômalas durante a execução.

Descrevemos exemplos típicos, detalhamos informações úteis para diagnóstico (como `Message`, `StackTrace` e `InnerException`) e explicamos o mecanismo de propagação e tratamento por meio de `try`, `catch` e `finally`, incluindo filtros com `when`. Também ressaltamos a diferença entre violações de contrato e situações esperadas do domínio, defendendo que cenários previsíveis sejam tratados com lógica de controle e padrões de retorno apropriados, enquanto exceções devem representar quebras de pressupostos, invariantes ou condições ambientais que inviabilizam a continuidade normal. Nesse enquadramento, a validação explícita de entradas e a escolha criteriosa do que lançar e do que capturar influenciam legibilidade, manutenção e capacidade de diagnóstico.

Na sequência, esta unidade vinculou robustez à disciplina de testes automatizados, situando-a no contexto de integração contínua e práticas de DevOps, nas quais a suíte de testes funciona como mecanismo de regressão para reduzir o risco de mudanças introduzirem falhas em funcionalidades já estabilizadas.

Apresentamos o xUnit como ferramenta central no ecossistema moderno de .NET, descrevendo sua forma de modelar testes por meio de atributos como `[Fact]` e `[Theory]`, o papel do runner na descoberta e na execução e a integração com o Gerenciador de Testes do Visual Studio. A exposição incluiu a lógica básica de escrita de testes e exemplificou um fluxo didático com um pequeno domínio de perfis em que implementamos a regra de match por curta mútua e demonstramos, por contraste, como um teste com expectativa equivocada falha, deixando claro que

o framework revela divergências entre o comportamento observado e o comportamento declarado como esperado.

Por fim, apresentamos os fundamentos de memória e a concorrência no ambiente gerenciado do .NET, argumentando que abstrações como GC e APIs de alto nível não eliminam decisões relevantes sobre representação de dados e sobre execução paralela e assíncrona.

Discutimos implicações práticas da distinção entre tipos por valor e tipos por referência, relacionando essas escolhas a padrões de alocação, custo de cópias, pressão sobre o GC e clareza de invariantes, além de apontar efeitos de boxing e de uso de `struct` e `class`.

No campo da concorrência, contrapomos o modelo de threads, como base de execução controlada pelo sistema operacional, ao modelo preferencial de Tasks no .NET, em que a TPL distribui unidades de trabalho sobre o thread pool. Explicamos o papel de `Task`, de `async/await` e do cancelamento cooperativo com `CancellationToken` para lidar com operações longas, sobretudo em cenários dominados por I/O, nos quais bloquear threads degrada responsividade.

Com isso, delimitamos um panorama que conecta tratamento de falhas, testes e compreensão do runtime como requisitos para software em C# que permaneça previsível, sustentável e com comportamento consistente sob evolução.



Exercícios

Questão 1. Em um serviço de pagamento em C#, um método `ProcessarPagamento()` pode lançar exceções do domínio (por exemplo, `PagamentoRecusadoException`) e, também, exceções inesperadas (como falhas de rede). O time quer tratar a recusa localmente, registrar o erro e impedir que erros inesperados sejam omitidos, mantendo o rastreo original para diagnóstico. Considerando as boas práticas de tratamento de exceções, qual alternativa é a mais coerente?

- A) Usar as exceções como caminho normal do fluxo, lançando e capturando exceções em cada validação, para simplificar os `if`.
- B) Capturar a `System.Exception` e não fazer nada no `catch`, para evitar que a aplicação encerre e para manter o sistema disponível.
- C) Capturar a exceção específica do domínio, registrar e encerrar o tratamento local. Para as exceções não previstas, registrar e relançar com `throw`, preservando o `StackTrace`.
- D) Capturar apenas `System.ApplicationException`, pois ela é a raiz recomendada para exceções de aplicação e cobre todos os casos.
- E) Substituir o mecanismo de exceções por códigos de retorno em todas as operações, pois o `try/catch` é sempre mais lento e deve ser evitado em qualquer situação.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: exceções não devem representar o fluxo normal de controle; elas sinalizam violações de contrato ou de pressupostos ou condições de execução anormais. Usá-las como substitutas de validações comuns tende a piorar a legibilidade e o desempenho.

B) Alternativa incorreta.

Justificativa: capturar e ignorar exceções dificulta a depuração e pode mascarar falhas relevantes, deixando o sistema em estado inconsistente e sem informação diagnóstica.

C) Alternativa correta.

Justificativa: tratar a exceção específica (por exemplo, a recusa de pagamento) permite uma resposta adequada ao caso esperado do domínio. Para as demais exceções, usar o relançamento com `throw`; após o registro evita sufocar erros inesperados e preserva o rastreo original, útil para investigação.

D) Alternativa incorreta.

Justificativa: exceções do domínio e do sistema derivam de `System.Exception`. Além disso, a prática comum é derivar exceções personalizadas diretamente de `Exception` em vez de depender de `ApplicationException` como base universal.

E) Alternativa incorreta.

Justificativa: códigos de retorno podem ser úteis em alguns contextos, mas não substituem o modelo de exceções do C# para condições excepcionais, que separa o código principal do código de tratamento e permite propagação automática pela pilha.

Questão 2. Uma aplicação C# precisa fazer uma operação demorada de I/O (por exemplo, uma requisição HTTP) sem bloquear a interface e deve permitir que o usuário cancele a operação. O time já usa `Task` e `async/await`, mas não tem certeza de como estruturar o cancelamento e como tratar o término por cancelamento. Considerando `Task`, `async/await` e o cancelamento cooperativo com `CancellationToken`, qual alternativa é a mais adequada?

- A) Criar uma thread manualmente para cada requisição e, no cancelamento, chamar um método de aborto forçado da thread, pois isso encerra o trabalho de forma imediata e segura.
- B) Iniciar a operação com `Task.Run`, ignorar `CancellationToken` e, no cancelamento, encerrar o aplicativo. Assim, evita-se ter de tratar exceções e estados intermediários.
- C) Fazer o método assíncrono aceitar um `CancellationToken`, repassar esse token às APIs assíncronas que o suportam (por exemplo, `GetAsync(..., ct)` e `Task.Delay(..., ct)`) e verificar o token em pontos adequados (por exemplo, com `ct.ThrowIfCancellationRequested()`). No chamador, usar `CancellationTokenSource.Cancel()` e tratar `OperationCanceledException` para encerrar de forma controlada.
- D) Para manter a interface responsiva, chamar `Task.WaitAll` e `Result` na thread da interface, pois essas chamadas aguardam mais rápido do que `await`.
- E) Envolver todo o método em `try/catch (Exception)` e, em qualquer erro ou cancelamento, suprimir a exceção silenciosamente. Assim, a aplicação nunca falha e o usuário não percebe interrupções.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: o modelo descrito no conteúdo é o de alto nível com `Task` e cancelamento cooperativo. Abortos forçados não representam o padrão recomendado e tendem a comprometer o encerramento limpo de operações e recursos.

B) Alternativa incorreta.

Justificativa: ignorar o token elimina o cancelamento cooperativo e fechar o aplicativo não é um tratamento do cancelamento, e sim uma interrupção abrupta do processo.

C) Alternativa correta.

Justificativa: a alternativa segue o fluxo típico: o chamador cria `CancellationTokenSource`, passa `cts.Token`, solicita o cancelamento via `cts.Cancel()` e o método assíncrono coopera respeitando o token, culminando em `OperationCanceledException`, que pode ser tratada no chamador para finalizar a operação adequadamente.

D) Alternativa incorreta.

Justificativa: bloquear com `WaitAll` ou `Result` contraria o objetivo de não bloquear a thread chamadora. O uso de `await` foi introduzido exatamente para aguardar sem travar a thread.

E) Alternativa incorreta.

Justificativa: capturar e suprimir exceções de forma genérica encobre falhas e dificulta o diagnóstico. Além disso, o cancelamento tem um fluxo próprio (com `OperationCanceledException`) que deve ser tratado de forma explícita quando o chamador precisa reagir ao cancelamento.

REFERÊNCIAS

- 2025 Stack Overflow developer survey. *Stack Overflow*, 2025. Disponível em: <https://tinyurl.com/32tv4y3t>. Acesso em: 21 nov. 2025.
- ALBAHARI, J.; ALBAHARI, B. *LINQ pocket reference*. Sebastopol: O'Reilly Media, 2008.
- ALBAHARI, J. *C# 10 in a nutshell: the definitive reference*. Sebastopol: O'Reilly Media, 2022.
- ALLS, J. *Clean code with C#*. 2. ed. Birmingham: Packt Publishing, 2023.
- ARAÚJO, E. C. *Orientação a objetos em C#: conceitos e implementações em .NET*. São Paulo: Casa do Código, 2017.
- BLOCH, J. *Effective Java*. 3. ed. Boston: Addison-Wesley, 2018.
- BOOCH, G. *et al. Object-oriented analysis and design with applications*. 3. ed. Boston: Addison-Wesley, 2007.
- CHACON, S.; STRAUB, B. *Pro Git*. 2. ed. Berkeley: Apress, 2014.
- CLEARY, S. *Concurrency in C# cookbook: asynchronous, parallel, and multithreaded programming*. 2. ed. Sebastopol: O'Reilly Media, 2019.
- CLYMER, A.; BLEWETT, R. *Pro asynchronous programming with .NET*. New York: Apress, 2013.
- COMPARAR as edições do Visual Studio. *Microsoft Visual Studio*, [s.d.]. Disponível em: <https://tinyurl.com/ym7j8c8d>. Acesso em: 26 nov. 2025.
- DEITEL, P.; DEITEL, H. *Visual C#: how to program*. 6. ed. Boston: Pearson, 2016.
- DOCUMENTAÇÃO da linguagem C#. *Microsoft Learn*, [s.d.]. Disponível em: <https://tinyurl.com/75zymhsd>. Acesso em: 6 jan. 2026.
- ECMA INTERNATIONAL. *ECMA-334: C# language specification*. Geneva: ECMA International, 2023. Disponível em: <https://tinyurl.com/3mya2btj>. Acesso em: 27 nov. 2025.
- ECMA INTERNATIONAL. *ECMA-335: common language infrastructure (CLI)*. Geneva: ECMA International, 2012. Disponível em: <https://tinyurl.com/5cstxdy7>. Acesso em: 27 nov. 2025.
- EVANS, E. *Domain-driven design: atacando as complexidades no coração do software*. 3. ed. Rio de Janeiro: Alta Books, 2015.
- FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps – building and scaling high performing technology organizations*. Portland: IT Revolution, 2018.

FOWLER, M. *Refatoração: aperfeiçoando o design de códigos existentes*. 2. ed. Porto Alegre: Novatec, 2020.

FREEMAN, A.; RATTZ, J. C. *Pro LINQ: language integrated query in C#* 2010. Berkeley: Apress, 2010.

GADDIS, T. *Starting out with visual C#*. 4. ed. Harlow: Pearson, 2017.

GAMMA, E. et al. *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2005.

GRIFFITHS, I. *Programming C# 10: build cloud, web, and desktop applications*. Sebastopol: O'Reilly Media, 2022.

GUNNERSON, E.; WIENHOLT, N. *A programmer's guide to C# 5.0*. 4. ed. Berkeley: Apress, 2012.

HALL, G. M. *Adaptive code via C#: agile coding with design patterns and SOLID principles*. 2. ed. Redmond: Microsoft Press, 2017.

HORSTMANN, C. S. *Object-oriented design & patterns*. 2. ed. Hoboken: John Wiley & Sons, 2006.

HUNT, D.; THOMAS, A. *O programador pragmático: de aprendiz a mestre*. Porto Alegre: Bookman, 2010.

KHORIKOV, V.; OSHEROVE, R. *The art of unit testing*. 3. ed. Shelter Island: Manning, 2024.

KIM, G. et al. *The DevOps handbook: how to create world-class agility, reliability, & security in technology organizations*. 2. ed. Portland: IT Revolution, 2021.

LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.

LIBERTY, J. *Programming C#*. 4. ed. Sebastopol: O'Reilly Media, 2005.

MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.

MARTIN, R. C. *Código limpo: habilidades práticas do Agile Software*. Rio de Janeiro: Alta Books, 2009.

MARTIN, R. C.; MARTIN, M. *Princípios, padrões e práticas ágeis em C#*. Porto Alegre: Bookman, 2011.

MESZAROS, G. *xUnit test patterns: refactoring test code*. Boston: Addison-Wesley, 2007.

MICHAELIS, M. *Essential C# 12.0*. 8. ed. Boston: Addison-Wesley, 2023.

PAINTER, S. J. *Functional programming with C#: create more supportable, robust, and testable code*. Sebastopol, CA: O'Reilly Media, 2023.



A series of horizontal lines for writing, consisting of 30 evenly spaced lines across the page.



Handwriting practice lines consisting of 30 horizontal lines. Each line is preceded by a small blue dot on the left margin, serving as a starting point for letter formation. The lines are evenly spaced and extend across the width of the page.



A series of horizontal lines for writing, consisting of 30 evenly spaced lines across the page.



Informações:
www.sepi.unip.br ou 0800 010 9000